

PostgreSQL: Top 44 Issues and Fixes — A Complete Field Guide

Who this is for: DBAs, backend engineers, and platform teams who operate PostgreSQL in production and need fast, trustworthy answers when things go wrong.

What you will find here: 44 real-world PostgreSQL problems, each with a clear explanation of the root cause, diagnostic queries, a concrete fix with example code, and a takeaway you can act on immediately.

Introduction: Why PostgreSQL Breaks (And How to Be Ready)

PostgreSQL is one of the most reliable databases ever built. It is also one of the most complex. In twenty-plus years of watching production systems, the same failure patterns appear again and again — not because teams are careless, but because PostgreSQL's internal machinery has a lot of moving parts, and the interactions between those parts are not always obvious until something catches fire at 2 a.m.

This guide exists to shorten that learning curve. Every issue in this list has been observed in production environments at companies running PostgreSQL at scale. The fixes are not theoretical. The queries you can copy directly into your `psql` session.

A few ground rules before we dive in:

1. Always test fixes in a staging environment first. Many remediation steps here — killing backends, running `VACUUM FREEZE`, dropping replication slots — are irreversible or have side effects.
 2. Version matters. Some features appeared in specific PostgreSQL versions. Version notes are called out where relevant.
 3. Statistics reset at restart. Many `pg_stat_*` queries return zero counts after a server restart. Wait for the system to accumulate data before drawing conclusions from zero readings.
-

Table of Contents

1. [Connection Exhaustion](#)
2. [Lock Contention and Blocking Chains](#)
3. [Deadlocks](#)
4. [Idle-in-Transaction Connections](#)
5. [Transaction ID Wraparound](#)
6. [Autovacuum Not Keeping Up](#)

7. [Table Bloat from Dead Tuples](#)
8. [Index Bloat](#)
9. [Missing Indexes Causing Sequential Scans](#)
10. [Unused Indexes Wasting Write Overhead](#)
11. [Slow Queries Due to Bad Planner Estimates](#)
12. [Stale Statistics After Bulk Loads](#)
13. [work_mem Too Low Causing Disk Spills](#)
14. [shared_buffers Undersized](#)
15. [Checkpoint Storms](#)
16. [WAL Bloat Filling Disk](#)
17. [Replication Lag on Standbys](#)
18. [Inactive Replication Slots](#)
19. [Hot Standby Conflict Cancellation](#)
20. [Connection Pooler Misconfiguration \(PgBouncer\)](#)
21. [Out-of-Memory \(OOM\) Kills](#)
22. [Disk Space Exhaustion](#)
23. [Temporary File Explosion](#)
24. [N+1 Query Patterns from ORMs](#)
25. [Partition Pruning Not Working](#)
26. [Parallel Query Not Used](#)
27. [Query Plan Regression After Upgrade](#)
28. [Index Not Used Due to Type Mismatch](#)
29. [Slow COUNT\(*\) on Large Tables](#)
30. [LIKE Queries Ignoring Indexes](#)
31. [JSON/JSONB Query Performance](#)
32. [Full-Text Search Performance](#)
33. [Sequence Exhaustion](#)
34. [Constraint Violations at Scale](#)
35. [Foreign Key Locking Overhead](#)
36. [Long-Running Schema Migrations](#)
37. [Table-Level Lock from ALTER TABLE](#)
38. [Connection Storms After Restart](#)
39. [Logical Replication Lag and Conflicts](#)
40. [Backup Failure from pg_dump](#)
41. [Point-in-Time Recovery Misconfiguration](#)
42. [SSL/TLS Connection Issues](#)
43. [Extension Conflicts and Version Mismatches](#)
44. [Corrupt Indexes and Data Pages](#)

Issue 1: Connection Exhaustion

The Problem

PostgreSQL allocates a fixed pool of connection slots set by `max_connections`. When all slots are taken, every new connection attempt fails immediately with:

```
FATAL: sorry, too many clients already
```

Unlike some databases, PostgreSQL does not queue connection requests — it rejects them outright. This makes connection exhaustion a hard outage, not a slowdown.

The root cause is almost never "too many users." It is almost always one of three things: no connection pooler (each application thread holds a dedicated backend process), a connection leak (the application opens connections and never closes them), or an idle-in-transaction pile-up (sessions are open but not working).

Diagnostic Queries

```
sql

-- Total usage and headroom
SELECT
  (SELECT setting::int FROM pg_settings WHERE name = 'max_connections') AS max_co
count(*) AS used_conn,
  (SELECT setting::int FROM pg_settings WHERE name = 'max_connections') - count(*)
round(100.0 * count(*) /
  (SELECT setting::int FROM pg_settings WHERE name = 'max_connections'), 1) AS
FROM pg_stat_activity;

-- Breakdown by application and state
SELECT
  application_name,
  state,
  count(*) AS cnt
FROM pg_stat_activity
WHERE pid != pg_backend_pid()
GROUP BY application_name, state
ORDER BY cnt DESC;
```

The Fix

Immediate — stop the bleeding:

```
sql

SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle'
      AND query_start < now() - interval '10 minutes'
      AND pid != pg_backend_pid();
```

Structural fix — deploy PgBouncer in transaction mode:

```
ini

# /etc/pgbouncer/pgbouncer.ini
[databases]
mydb = host=127.0.0.1 port=5432 dbname=mydb

[pgbouncer]
pool_mode = transaction
max_client_conn = 5000
default_pool_size = 25
min_pool_size = 5
reserve_pool_size = 5
server_idle_timeout = 600
```

Lower max_connections on the server and rely on the pooler:

```
ini

# postgresql.conf
max_connections = 100
superuser_reserved_connections = 3
```

Takeaway

Connection exhaustion is a pooler problem, not a `max_connections` problem. Raising `max_connections` increases shared memory consumption and context-switching overhead without fixing the root cause. Deploy PgBouncer in transaction mode and keep your server-side connection count under 200.

Issue 2: Lock Contention and Blocking Chains

The Problem

THE PROBLEM

PostgreSQL uses MVCC but still requires locks for writes. When two transactions try to modify the same row, or when a DDL statement needs an `AccessExclusiveLock`, one transaction must wait. If the blocker is itself waiting on something else, you get a chain that can paralyse your entire application.

The insidious part: the blocking chain is not visible in application logs. The application just hangs. Database CPU drops to near zero. DBA gets paged.

Diagnostic Queries

```
sql

-- Full blocking chain with wait times (PostgreSQL 9.6+)
SELECT
    blocked.pid           AS waiting_pid,
    blocked.username      AS waiting_user,
    blocked.query         AS waiting_query,
    blocking.pid          AS blocking_pid,
    blocking.username     AS blocking_user,
    blocking.query        AS blocking_query,
    now() - blocked.query_start AS wait_duration
FROM pg_stat_activity AS blocked
JOIN pg_stat_activity AS blocking
    ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0
ORDER BY wait_duration DESC;

-- Lock mode detail
SELECT
    pid,
    relation::regclass AS table_name,
    mode,
    granted,
    locktype
FROM pg_locks
WHERE relation IS NOT NULL
ORDER BY granted, pid;
```

The Fix

Immediate — terminate the root blocker:

```
sql

SELECT pg_terminate_backend(12345); -- use blocking_pid from above
```

Keep transactions short — move work outside the transaction:

Keep transactions short — move work outside the transaction.

```
sql

-- BAD: transaction stays open while application processes results
BEGIN;
SELECT * FROM orders WHERE status = 'pending';
-- ... application does work for 30 seconds ...
UPDATE orders SET status = 'processing' WHERE id = 42;
COMMIT;

-- GOOD: fetch data outside the transaction; only lock during the write
SELECT * FROM orders WHERE status = 'pending';
-- ... application does work ...
BEGIN;
UPDATE orders SET status = 'processing' WHERE id = 42;
COMMIT;
```

Use `SELECT ... FOR UPDATE SKIP LOCKED` for queue patterns:

```
sql

SELECT id, payload
FROM job_queue
WHERE status = 'pending'
ORDER BY created_at
LIMIT 1
FOR UPDATE SKIP LOCKED;
```

Set `lock_timeout` to prevent indefinite waits:

```
ini

# postgresql.conf or per session
lock_timeout = '5s'
```

Takeaway

The root of most blocking incidents is a long-running transaction holding a lock. Use `pg_blocking_pids()` to find the root blocker immediately. Set `lock_timeout` on all application connections to guarantee no query waits forever. Keep transactions as short as possible — ideally under one second.

Issue 3: Deadlocks

The Problem

A deadlock happens when transaction A holds a lock that transaction B needs, and transaction B holds a lock that transaction A needs. PostgreSQL detects this automatically and cancels one transaction with:

```
ERROR: deadlock detected
DETAIL: Process 1234 waits for ShareLock on transaction 5678;
       blocked by process 5679.
```

Deadlocks are not random. They follow predictable patterns in application code.

Diagnostic Queries

```
sql

-- Count deadlocks per database since last stats reset
SELECT datname, deadlocks
FROM pg_stat_database
ORDER BY deadlocks DESC;
```

```
ini

# Enable deadlock logging in postgresql.conf
log_lock_waits = on
deadlock_timeout = 1s
```

The Fix

Consistent lock ordering — the most common fix:

```
sql

-- Transaction 1 and 2 both update rows 1 and 2 in opposite orders = DEADLOCK
-- Fix: always lock rows in the same order

-- Use ORDER BY in multi-row updates
UPDATE accounts SET balance = balance - 100
WHERE id = ANY(ARRAY[1, 2])
ORDER BY id; -- consistent ordering prevents deadlock
```

Use advisory locks for complex ordering:

```
sql

SELECT pg_advisory_xact_lock(account_id)
```

```
FROM (VALUES (1), (2)) AS t(account_id)
ORDER BY account_id; -- acquire in a consistent order

UPDATE accounts SET ... WHERE id IN (1, 2);
```

Retry deadlocked transactions in application code:

```
python

import psycopg2
from psycopg2 import errors
import time, random

def execute_with_retry(conn, fn, max_retries=3):
    for attempt in range(max_retries):
        try:
            with conn.cursor() as cur:
                fn(cur)
            conn.commit()
            return
        except errors.DeadlockDetected:
            conn.rollback()
            if attempt == max_retries - 1:
                raise
            time.sleep(0.1 * (2 ** attempt) + random.uniform(0, 0.1))
```

Takeaway

Deadlocks are always an application design issue. Lock rows in a consistent, deterministic order across all code paths. Enable `log_lock_waits` so deadlocks appear in your log aggregator. Build retry logic with exponential backoff for any transaction touching multiple rows that could conflict with concurrent transactions.

Issue 4: Idle-in-Transaction Connections

The Problem

An "idle in transaction" session has opened a `BEGIN` but is not currently executing a query. It is holding all locks acquired so far and preventing `VACUUM` from cleaning rows it has read. A single stuck idle-in-transaction session can block `VACUUM`, hold row-level locks that block writes, and consume a connection slot.

Diagnostic Queries


```
sql

SELECT
    pid,
    username,
    application_name,
    state,
    now() - state_change      AS idle_in_txn_duration,
    left(query, 100)         AS last_query
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'idle in transaction (aborted)')
ORDER BY idle_in_txn_duration DESC;
```

The Fix

Immediate — kill stuck sessions:

```
sql

SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'idle in transaction (aborted)')
      AND state_change < now() - interval '5 minutes';
```

Structural fix — set the timeout (PostgreSQL 9.6+):

```
ini

# postgresql.conf
idle_in_transaction_session_timeout = '5min'
```

Application fix — use context managers:

```
python

# GOOD: transaction always ends at block exit regardless of exceptions
with conn:
    with conn.cursor() as cur:
        cur.execute("UPDATE cart SET ...")
# Transaction committed or rolled back here — never left open
```

Takeaway

Set `idle_in_transaction_session_timeout` to 2-5 minutes in `postgresql.conf`. This is the single cheapest configuration change that prevents an entire class of production incidents. Never open a transaction in application code and then wait for user input, a network call, or any slow operation inside it.

Issue 5: Transaction ID Wraparound

The Problem

PostgreSQL assigns a 32-bit transaction ID (XID) to every transaction. When the counter approaches 2 billion, PostgreSQL enters a failsafe mode and — if not addressed — shuts down with:

```
ERROR: database is not accepting commands to avoid wraparound data loss
```

This is not theoretical. It has taken down production databases at major companies. The preventive mechanism is `VACUUM FREEZE`, which marks old tuples as permanently visible so their XIDs can be reused.

The danger metric is `age(relfrozenxid)` for each table. When this approaches `autovacuum_freeze_max_age` (default 200 million), autovacuum schedules an aggressive freeze. Approaching 1.5 billion triggers a read-only failsafe.

Diagnostic Queries

```
sql

-- Tables with the oldest XIDs (most at-risk)
SELECT
    n.nspname                AS schema,
    c.relname                 AS table_name,
    age(c.relfrozenxid)       AS xid_age,
    pg_size_pretty(pg_table_size(c.oid)) AS table_size,
    2000000000 - age(c.relfrozenxid)    AS txns_until_failsafe
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
    AND n.nspname NOT IN ('pg_catalog', 'information_schema')
ORDER BY xid_age DESC
LIMIT 20;

-- Database-level XID age
SELECT datname, age(datfrozenxid) AS db_xid_age
FROM pg_database
ORDER BY age(datfrozenxid) DESC;
```

The Fix

the fix

Alert thresholds: Alert at 500 million, emergency at 1 billion.

Manual freeze for at-risk tables:

```
sql

-- Run during low-traffic window
VACUUM FREEZE VERBOSE my_large_table;
```

Tune autovacuum to freeze more aggressively:

```
ini

# postgresql.conf
autovacuum_freeze_max_age = 200000000
vacuum_freeze_table_age   = 150000000

# Per-table override for rarely-updated tables
-- ALTER TABLE rarely_updated SET (autovacuum_freeze_max_age = 100000000);
```

Takeaway

XID wraparound is the PostgreSQL equivalent of a self-inflicted hard shutdown. Monitor `age(relfrozenxid)` on every table. Never let autovacuum fall behind on long-lived, rarely-updated tables — they accumulate XID age silently. Alert at 500 million. Treat 1 billion as a PO incident.

[issue-6](#)

Issue 6: Autovacuum Not Keeping Up

The Problem

Autovacuum is PostgreSQL's background process for reclaiming dead tuples and updating planner statistics. When it cannot keep up with write load, tables bloat, plans degrade, and XID age creeps toward wraparound.

Signs autovacuum is falling behind: tables with millions of `n_dead_tup`, growing table sizes with no corresponding data growth, query plans switching from index scans to sequential scans, and XID age rising faster than autovacuum can freeze it.

Diagnostic Queries

```
sql
```

```

-- Tables with the most dead tuples and stale autovacuum
SELECT
    schemaname,
    relname                                AS table_name,
    n_dead_tup,
    n_live_tup,
    round(n_dead_tup::numeric / nullif(n_live_tup, 0), 3) AS dead_ratio,
    last_autovacuum,
    last_autoanalyze,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||relname)) AS size
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC
LIMIT 20;

-- Active autovacuum workers
SELECT pid, now() - query_start AS runtime, query
FROM pg_stat_activity
WHERE query LIKE 'autovacuum:%'
ORDER BY runtime DESC;

```

The Fix

Immediate — manual vacuum:

```

sql

VACUUM ANALYZE my_bloated_table;
VACUUM (ANALYZE, VERBOSE) my_bloated_table;

```

Tune autovacuum globally for write-heavy workloads:

```

ini

# postgresql.conf
autovacuum_max_workers = 6
autovacuum_vacuum_scale_factor = 0.01      # was 0.2
autovacuum_analyze_scale_factor = 0.005    # was 0.1
autovacuum_vacuum_cost_delay = 2ms
autovacuum_vacuum_cost_limit = 400

```

Per-table override for hot tables:

sql

```
ALTER TABLE events SET (  
    autovacuum_vacuum_scale_factor = 0.01,  
    autovacuum_analyze_scale_factor = 0.005,  
    autovacuum_vacuum_cost_delay = 0,  
    autovacuum_vacuum_cost_limit = 1000  
);
```

Takeaway

Default autovacuum settings were designed for modest workloads. Any table receiving more than a few thousand writes per minute needs tuned per-table autovacuum parameters. Lower scale factors, increase worker count, reduce cost delay. Monitor `n_dead_tup` and `last_autovacuum` in your metrics pipeline — not just at incident time.

[issue-7](#)

Issue 7: Table Bloat from Dead Tuples

The Problem

PostgreSQL's MVCC architecture never overwrites rows in place. Every `UPDATE` writes a new row version; every `DELETE` marks the old row as dead. `VACUUM` reclaims dead rows' space for reuse within the same file but does not return it to the OS by default. `VACUUM FULL` returns space to the OS but requires an `AccessExclusiveLock` blocking all table access.

A bloated table wastes disk space, forces sequential scans to read unnecessary pages, and degrades cache hit ratio.

Diagnostic Queries

sql

```

-- Approximate bloat without extension
SELECT
    relname,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size,
    pg_size_pretty(pg_relation_size(c.oid))      AS table_size,
    n_dead_tup,
    n_live_tup,
    round(100.0 * n_dead_tup / nullif(n_live_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables
JOIN pg_class c USING (relname)
ORDER BY pg_total_relation_size(c.oid) DESC
LIMIT 20;

-- Precise bloat measurement (requires extension)
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT
    relname,
    pg_size_pretty(relation_size) AS size,
    dead_tuple_percent,
    pg_size_pretty(dead_tuple_len) AS dead_space
FROM pgstattuple('my_large_table');

```

The Fix

Option 1 — `pg_repack` (online, preferred for production):

```

bash

# Install
apt install postgresql-14-repack

# Repack a single table online — no extended locks
pg_repack -h localhost -U postgres -d mydb -t my_bloated_table

```

```
# Repack all user tables
pg_repack -h localhost -U postgres -d mydb
```

Option 2 — `VACUUM FULL` (requires maintenance window):

```
sql

VACUUM FULL ANALYZE my_bloated_table;
```

Option 3 — partition and DROP for time-series tables:

```
sql

CREATE TABLE events_2024_01 PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Later: instant, no bloat
DROP TABLE events_2023_01;
```

Takeaway

Regular `VACUUM` prevents bloat accumulation. When bloat has already occurred, use `pg_repack` for production tables — it runs online with minimal locking. For append-heavy tables, switch to range partitioning by time so old data can be dropped instantaneously. Never run `VACUUM FULL` on a production table without a maintenance window.

Issue 8: Index Bloat

The Problem

B-tree indexes also accumulate bloat. When rows are deleted or updated, index entries are marked dead but not immediately reclaimed. Index bloat causes larger index files consuming more cache space, slower index scans, and increased write amplification.

Diagnostic Queries

```
sql

-- Index sizes and usage
SELECT
    schemaname,
    tablename,
    indexname,
```

```

pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
idx_scan,
idx_tup_read
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 20;

-- Detailed fragmentation (requires pgstattuple)
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT
    indexrelname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS size,
    leaf_fragmentation,
    avg_leaf_density
FROM pg_stat_user_indexes
CROSS JOIN LATERAL pgstatindex(indexrelname) AS s
ORDER BY leaf_fragmentation DESC NULLS LAST
LIMIT 10;

```

The Fix

REINDEX CONCURRENTLY — rebuild without blocking (PostgreSQL 12+):

```

sql

REINDEX INDEX CONCURRENTLY my_table_idx;
REINDEX TABLE CONCURRENTLY my_table;

```

Old method for pre-12 versions:

```

sql

CREATE INDEX CONCURRENTLY my_table_idx_new ON my_table(column_a);
DROP INDEX CONCURRENTLY my_table_idx;
ALTER INDEX my_table_idx_new RENAME TO my_table_idx;

```

Tune **fillfactor** to reduce future bloat:

```

sql

-- Leave 30% free space in each page for HOT updates
ALTER TABLE my_table SET (fillfactor = 70);
CREATE INDEX my_idx ON my_table(col) WITH (fillfactor = 70);

```

Takeaway

Schedule **REINDEX CONCURRENTLY** on your largest indexes quarterly, or more often on high-write tables. Set **fillfactor** below 100 on tables with frequent in-place updates to

high write tables. Set `(fillfactor)` below 100 on tables with frequent in-place updates to enable HOT (Heap-Only Tuple) updates that avoid index churn. Monitor index sizes alongside table sizes in your metrics dashboard.

Issue 9: Missing Indexes Causing Sequential Scans

The Problem

A sequential scan reads every page of a table regardless of how many rows match the query. On a table with millions of rows, a missing index can turn a 1-millisecond lookup into a 10-second full scan. The query planner chooses sequential scans when no index exists on the filtered columns, statistics are stale, the predicate is on a function result, or a type mismatch prevents index use.

Diagnostic Queries

```
sql

-- Tables with high sequential scan counts
SELECT
    schemaname,
    relname,
    seq_scan,
    idx_scan,
    n_live_tup,
    round(100.0 * seq_scan / nullif(seq_scan + idx_scan, 0), 1) AS pct_seq
FROM pg_stat_user_tables
WHERE n_live_tup > 10000
    AND seq_scan > 100
ORDER BY seq_scan DESC
LIMIT 20;
```

```
sql

-- Always use EXPLAIN to confirm the scan type
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 12345;
```

The Fix

Create the missing index:

```
sql
```

```
-- Single-column
CREATE INDEX CONCURRENTLY idx_orders_customer_id
  ON orders(customer_id);

-- Composite for filter + sort
CREATE INDEX CONCURRENTLY idx_orders_customer_status
  ON orders(customer_id, status, created_at DESC);

-- Partial index — much smaller, faster for targeted queries
CREATE INDEX CONCURRENTLY idx_orders_pending
  ON orders(created_at)
  WHERE status = 'pending';
```

Verify the index is used:

```
sql

EXPLAIN (ANALYZE)
SELECT * FROM orders WHERE customer_id = 12345;
-- Should show: Index Scan using idx_orders_customer_id
```

Run ANALYZE after bulk loads:

```
sql

ANALYZE orders;
```

Takeaway

Monitor `pg_stat_user_tables` for tables where `seq_scan` is growing rapidly. Use `EXPLAIN (ANALYZE, BUFFERS)` to confirm every slow query's scan type. Create partial indexes for common filtered patterns — they are smaller, faster, and updated less often than full indexes. Never add an index without verifying with `EXPLAIN` that the planner actually uses it.

[issue-10](#)

Issue 10: Unused Indexes Wasting Write Overhead

The Problem

Every index must be updated on every `INSERT`, `UPDATE`, and `DELETE` that touches indexed columns. An unused index is pure write overhead — it costs I/O and CPU on every write and

occupies disk space and cache memory while providing zero read benefit.

Teams accumulate unused indexes by adding them to fix specific slow queries, copying schemas between environments, or adding them "just in case."

Diagnostic Queries

```
sql

-- Indexes never scanned since last stats reset
SELECT
  schemaname,
  relname                                AS table_name,
  indexrelname                           AS index_name,
  idx_scan                               AS times_used,
  pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
      AND schemaname NOT IN ('pg_catalog', 'pg_toast')
ORDER BY pg_relation_size(indexrelid) DESC;
```

The Fix

Important caveats before dropping:

- Wait at least a week on a running server; statistics reset after restart
- Check periodic jobs and reporting queries that run infrequently
- Never drop unique indexes — they enforce constraints even if not scanned for reads

```
sql

-- Safe: DROP CONCURRENTLY does not block reads or writes
DROP INDEX CONCURRENTLY idx_orders_old_status;
```

Test impact before dropping — disable via planner settings:

```
sql

-- Temporarily tell planner to ignore index scans
SET enable_indexscan = off;
SET enable_bitmapscan = off;
-- Run your workload and check for regressions
SET enable_indexscan = on;
SET enable_bitmapscan = on;
```

Takeaway

Indexes are a double-edged sword. While they can significantly speed up reads, they also occupy disk space and cache memory while providing zero read benefit.

Run the unused-index query monthly. Anything with `(idx_scan = 0)` for more than two weeks on a server that has not been restarted is a candidate for removal. Unused indexes on high-write tables can reduce write throughput by 5-15%. In microservices, each service should own only the indexes its own queries actually use.

Issue 11: Slow Queries Due to Bad Planner Estimates

The Problem

The PostgreSQL query planner uses statistics collected by `ANALYZE` to estimate how many rows each step of a query will return. When estimates are wrong — because statistics are stale, column values are highly skewed, or correlated columns confuse the planner — it chooses bad plans.

Classic symptoms: a query that used to take 10 ms suddenly takes 10 seconds after data distribution changed, the planner choosing a sequential scan when an index would be faster, or a nested loop chosen for a large join that should be a hash join.

Diagnostic Queries

```
sql

-- Compare actual vs estimated rows
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT o.*, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'pending'
      AND o.created_at > now() - interval '7 days';
-- Look for: "rows=5000 (actual rows=1234567)" = bad estimate

-- Inspect current column statistics
SELECT
  attname,
  n_distinct,
  correlation,
  most_common_vals,
  most_common_freqs
FROM pg_stats
WHERE tablename = 'orders'
      AND attname = 'status';
```

The Fix

Update statistics and increase the statistics target for skewed columns:

```
sql

ANALYZE orders;

-- High-cardinality or skewed columns need more samples (default: 100)
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 500;
ANALYZE orders;
```

Use extended statistics for correlated columns (PostgreSQL 10+):

```
sql

-- If queries filter on both (country, city), which are correlated:
CREATE STATISTICS orders_country_city ON country, city FROM orders;
ANALYZE orders;
```

Force a plan for debugging (temporary only):

```
sql

SET enable_hashjoin = off; -- forces nested loop
EXPLAIN (ANALYZE) SELECT ...;
SET enable_hashjoin = on;
```

Takeaway

Bad planner estimates are always a data or statistics problem. Increase statistics targets on skewed columns, use extended statistics for correlated column combinations, and always run `ANALYZE` after bulk loads. A 10x discrepancy between estimated and actual rows in `EXPLAIN ANALYZE` output is a red flag that demands investigation.

Issue 12: Stale Statistics After Bulk Loads

The Problem

After a large bulk load (`COPY`, `INSERT ... SELECT`, or an ETL job), table statistics used by the query planner may be dramatically wrong. Autovacuum's `ANALYZE` trigger fires when 20% of table rows have changed — but if you loaded 10 million rows into a previously empty table, it may fire too late, or not at all if autovacuum is busy elsewhere. The result is that queries against the freshly loaded table use plans based on empty or wrong statistics, often choosing sequential scans or bad join orders.

Diagnostic Queries

```
sql

-- Check when statistics were last collected
SELECT
    relname,
    last_analyze,
    last_autoanalyze,
    n_live_tup,
    n_dead_tup,
    analyze_count
FROM pg_stat_user_tables
WHERE relname = 'my_loaded_table';
```

The Fix

Always run **ANALYZE** explicitly after bulk loads:

```
sql

COPY staging_events FROM '/data/events.csv' WITH (FORMAT csv, HEADER true);
ANALYZE staging_events;
```

Use **COPY** instead of row-by-row **INSERT**:

```
sql

-- COPY is orders of magnitude faster and generates less WAL
COPY orders (id, customer_id, amount, created_at)
FROM '/tmp/orders_export.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ',');
```

Complete bulk-load workflow:

```
sql

BEGIN;
ALTER TABLE staging_events DISABLE TRIGGER ALL;

COPY staging_events FROM '/data/events_2024.csv'
    WITH (FORMAT csv, HEADER true);

ALTER TABLE staging_events ENABLE TRIGGER ALL;
COMMIT;
```

```
-- Always analyze after
VACUUM ANALYZE staging_events;

-- Verify the plan is sane
EXPLAIN (ANALYZE)
SELECT count(*) FROM staging_events WHERE event_type = 'click';
```

Tune autovacuum thresholds for batch-loaded tables:

```
sql

ALTER TABLE staging_events SET (
    autovacuum_analyze_scale_factor = 0.01,
    autovacuum_analyze_threshold = 1000
);
```

Takeaway

`ANALYZE` is fast. Running it explicitly after every bulk load costs almost nothing and prevents an entire class of plan-degradation incidents. Make `VACUUM ANALYZE <table>` the last step in every ETL script, migration, and data-load procedure.

Issue 13: work_mem Too Low Causing Disk Spills

The Problem

`work_mem` is the amount of memory PostgreSQL allocates per sort or hash operation per query node. When a sort or hash join cannot fit in `work_mem`, PostgreSQL spills to disk using temporary files. A single complex query can spawn dozens of sort or hash operations — each consuming up to `work_mem` bytes.

Low `work_mem` causes slow sorts, slow GROUP BY operations, slow hash joins, temp files accumulating in `pg_temp`, and high disk I/O during batch queries.

Diagnostic Queries

```
sql

-- Queries currently spilling to disk (requires pg_stat_statements)
SELECT
    left(query, 120),
    calls,
    round(mean_exec_time::numeric, 2) AS avg_ms,
```

```
temp_blks_written,  
temp_blks_read  
FROM pg_stat_statements  
WHERE temp_blks_written > 0  
ORDER BY temp_blks_written DESC  
LIMIT 10;
```

In `EXPLAIN ANALYZE` output, look for:

```
Sort Method: external merge  Disk: 42768kB    <-- spill to disk (bad)  
Sort Method: quicksort      Memory: 4096kB     <-- in memory (good)
```

The Fix

Increase `work_mem` carefully:

`work_mem` is per-operation, per-connection. Setting it globally high can cause OOM if many sessions run complex queries simultaneously.

```
ini  
  
# postgresql.conf - conservative global default  
work_mem = 64MB
```

Per-role overrides for analytics users:

```
sql  
  
ALTER ROLE analytics_user SET work_mem = '256MB';  
ALTER ROLE app_user SET work_mem = '32MB';
```

Temporary override for a specific heavy query:

```
sql  
  
SET work_mem = '512MB';  
SELECT ... complex aggregation ...;  
RESET work_mem;
```

Rule of thumb for global setting:

$$\text{work_mem} \approx (\text{Total RAM} \times 0.25) \div (\text{max_connections} \times \text{avg_sort_ops_per_query})$$

For 64GB RAM, 100 connections, 2 sort ops per query: ~80MB.

Takeaway

Do not raise `work_mem` globally without calculating worst-case memory usage. Set a conservative global value (32-64MB) and use `ALTER ROLE` to give higher values to batch and analytics users. Monitor `pg_stat_statements.temp_blks_written` — any non-zero value means queries are spilling to disk unnecessarily.

Issue 14: `shared_buffers` Undersized

The Problem

`shared_buffers` is PostgreSQL's internal page cache. It is the first place PostgreSQL looks for data before going to the OS page cache or disk. If it is too small, every query reads from the OS cache or disk, dramatically increasing I/O and latency. The default `shared_buffers = 128MB` is appropriate for a development laptop, not a production server with 64GB RAM.

Diagnostic Queries

```
sql

-- Overall cache hit ratio (should be > 99% on OLTP)
SELECT
    sum(heap_blks_hit) AS cache_hits,
    sum(heap_blks_read) AS disk_reads,
    round(
        100.0 * sum(heap_blks_hit)
        / nullif(sum(heap_blks_hit + heap_blks_read), 0), 2
    ) AS cache_hit_pct
FROM pg_statio_user_tables;

-- Per-table cache hit ratio
SELECT
    relname,
    heap_blks_hit,
    heap_blks_read,
    round(100.0 * heap_blks_hit
        / nullif(heap_blks_hit + heap_blks_read, 0), 2) AS hit_pct
FROM pg_statio_user_tables
WHERE heap_blks_hit + heap_blks_read > 0
ORDER BY heap_blks_read DESC
LIMIT 20;
```

The Fix

Set `shared_buffers` to 25% of RAM:

```
ini

# postgresql.conf
shared_buffers = 16GB          # 25% of 64GB total RAM
effective_cache_size = 48GB    # 75% of 64GB – planner hint only, allocates nothing
```

Enable huge pages on Linux for large `shared_buffers`:

```
ini

# postgresql.conf
huge_pages = on

# /etc/sysctl.conf
vm.nr_hugepages = 8192    # 8192 × 2MB = 16GB of huge pages
```

Monitor after change:

```
bash

# After restart, watch for swap usage – sign that shared_buffers is too large
vmstat 1 10
free -h
```

Takeaway

Set `shared_buffers` to 25% of total RAM as a baseline. Set `effective_cache_size` to 75% of total RAM. If the cache hit ratio is below 99% and you have free RAM, increase `shared_buffers`. On Linux with large `shared_buffers`, enable huge pages to reduce TLB pressure. A restart is required for changes to take effect.

Issue 15: Checkpoint Storms

The Problem

PostgreSQL writes all changes to WAL first, then periodically flushes dirty `shared_buffers` pages to data files in a process called a checkpoint. If checkpoints happen too frequently,

pages to data files in a process called a checkpoint. If checkpoints happen too frequently — due to high write volume — they cause spikes of disk I/O that saturate storage, query latency spikes, and write amplification.

Signs of checkpoint storms in logs:

```
LOG: checkpoint complete: wrote 45231 buffers (27.6%); duration: 42.321 s
WARNING: checkpoints are occurring too frequently (26 seconds apart)
```

Diagnostic Queries

```
sql

-- Checkpoint statistics
SELECT
    checkpoints_timed,
    checkpoints_req,           -- forced by WAL segments filling up
    checkpoint_write_time,
    checkpoint_sync_time,
    buffers_checkpoint,
    buffers_clean,
    maxwritten_clean,
    buffers_backend_fsync,
    stats_reset
FROM pg_stat_bgwriter;
```

High `checkpoints_req` vs `checkpoints_timed` means WAL is filling up faster than checkpoints can run.

The Fix

```
ini

# postgresql.conf

# Increase WAL buffer before forced checkpoint
max_wal_size = 8GB           # default: 1GB
min_wal_size = 1GB

# Spread I/O writes over 90% of checkpoint interval
```

```
checkpoint_completion_target = 0.9      # default: 0.5

# Allow longer between automatic checkpoints
checkpoint_timeout = 15min              # default: 5min
```

Match cost settings to your storage type:

```
ini

# NVMe SSD
random_page_cost = 1.1
seq_page_cost = 1.0

# Spinning disk
random_page_cost = 4.0
seq_page_cost = 1.0
```

Takeaway

If you see `checkpoints_req` growing, your WAL is filling faster than checkpoints can run. Increase `max_wal_size`, set `checkpoint_completion_target = 0.9`, and extend `checkpoint_timeout`. These three changes together dramatically smooth checkpoint I/O. A healthy system has mostly `checkpoints_timed`, not `checkpoints_req`.

Issue 16: WAL Bloat Filling Disk

The Problem

WAL files accumulate on disk when inactive replication slots are not consumed (the most dangerous cause), `wal_keep_size` is set too high, archiving is stalled, or high write volume outpaces checkpoints. WAL filling the disk is an emergency — PostgreSQL cannot accept new writes.

Diagnostic Queries

```
sql

-- Current WAL directory size
SELECT pg_size_pretty(sum(size)) AS wal_size
FROM pg_ls_waldir();

-- Replication slots and their WAL retention (the most common culprit)
SELECT
    slot_name
```

```
slot_name,  
active,  
confirmed_flush_lsn,  
pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS wal_retained  
FROM pg_replication_slots  
ORDER BY (pg_current_wal_lsn() - confirmed_flush_lsn) DESC NULLS LAST;
```

The Fix

Drop inactive replication slots (most common emergency fix):

sql

```
SELECT pg_drop_replication_slot('my_stale_slot');  
  
-- Batch drop all inactive slots  
SELECT pg_drop_replication_slot(slot_name)  
FROM pg_replication_slots  
WHERE active = false;
```

Set a WAL retention cap on slots (PostgreSQL 13+):

ini

```
# postgresql.conf  
max_slot_wal_keep_size = 10GB
```

After dropping slots, reduce wal_keep_size:

ini

```
# postgresql.conf  
wal_keep_size = 1GB    # or 0 if using archiving  
# Then: SELECT pg_reload_conf();
```

Takeaway

Inactive replication slots are the single most common cause of WAL filling disk in production. Set `max_slot_wal_keep_size` in PostgreSQL 13+ to cap retention. Alert when any replication slot's WAL retention exceeds 10GB. If disk fills due to WAL, drop inactive slots first — they are the likeliest culprit and dropping them is safe if no

consumer needs them.

Issue 17: Replication Lag on Standbys

The Problem

Streaming replication ships WAL from the primary to standbys, which replay it. When the standby cannot replay WAL as fast as the primary generates it, replication lag accumulates. Consequences include read queries returning stale data, data loss on failover proportional to lag, and eventual standby disconnection if lag grows indefinitely.

Diagnostic Queries

```
sql

-- On the primary: lag per connected standby
SELECT
  application_name,
  client_addr,
  state,
  (sent_lsn - replay_lsn) AS bytes_lag,
  write_lag,
  flush_lag,
  replay_lag
FROM pg_stat_replication
ORDER BY bytes_lag DESC NULLS LAST;

-- On the standby: lag from its own perspective
SELECT
  now() - pg_last_xact_replay_timestamp() AS replication_lag,
  pg_is_in_recovery() AS is_standby,
  pg_last_wal_receive_lsn() AS received_lsn,
  pg_last_wal_replay_lsn() AS replayed_lsn;
```

The Fix

Diagnose where lag accumulates:

```
sent_lsn - write_lsn → network bottleneck
```

sent_lsn - write_lsn → network bottleneck
write_lsn - flush_lsn → standby I/O bottleneck
flush_lsn - replay_lsn → standby CPU/WAL replay bottleneck

Enable parallel WAL replay (PostgreSQL 15+):

```
ini

# standby's postgresql.conf
recovery_parallelism = 4
```

Compress WAL to reduce network transfer:

```
ini

# primary's postgresql.conf
wal_compression = on
```

Enable `hot_standby_feedback` to reduce conflict cancellation:

```
ini

# standby's postgresql.conf
hot_standby_feedback = on
```

Takeaway

Replication lag is almost always an I/O or network bottleneck on the standby. Use the sent/write/flush/replay LSN chain to pinpoint where lag accumulates. Enable `wal_compression` to reduce network transfer. On PostgreSQL 15+, enable parallel WAL replay for high-write primaries. Alert when lag exceeds your RPO (recovery point objective).

Issue 18: Inactive Replication Slots

The Problem

A replication slot created for a standby or logical consumer but never cleaned up will silently retain all WAL generated since it was last consumed. Common causes include a decommissioned standby whose slot was not dropped, a deleted logical replication subscriber without `DROP SUBSCRIPTION`, or a CDC connector (Debezium, etc.) that crashed and was not restarted.

Diagnostic Queries

```
sql

-- All slots: active vs inactive, with WAL retained
SELECT
    slot_name,
    slot_type,
    database,
    active,
    active_pid,
    pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS wal_retained,
    plugin
FROM pg_replication_slots
ORDER BY (pg_current_wal_lsn() - confirmed_flush_lsn) DESC NULLS LAST;
```

The Fix

```
sql

-- Drop an unused physical slot
SELECT pg_drop_replication_slot('old_standby_slot');

-- Drop an unused logical slot
SELECT pg_drop_replication_slot('my_cdc_slot');

-- On the subscriber: fix an inactive subscription
ALTER SUBSCRIPTION my_sub ENABLE;
```

Monitoring query — alert if any inactive slot retains > 1GB:

```
sql

SELECT slot_name,
    pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS retained
FROM pg_replication_slots
WHERE active = false
    AND (pg_current_wal_lsn() - confirmed_flush_lsn) > 1073741824;
```

Takeaway

Treat replication slot creation as a lifecycle event that requires cleanup. Any process that creates a slot must have a corresponding cleanup step when it exits. Set

`max_slot_wal_keep_size` to cap worst-case damage. Monitor slot WAL retention in

your metrics pipeline — an inactive slot growing at your write volume is a ticking clock.

Issue 19: Hot Standby Conflict Cancellation

The Problem

Queries on a hot standby can be cancelled by the WAL replay process when replayed WAL would conflict with an active query. The error looks like:

```
ERROR: canceling statement due to conflict with recovery
DETAIL: User query might have needed to see row versions that must be removed.
```

This happens because a read query on the standby is reading rows that the primary has since vacuumed. When WAL replay tries to apply the vacuum, it conflicts with the standby query and cancels it.

Diagnostic Queries

```
sql

-- On the standby: count conflict cancellations per database
SELECT datname, conflicts FROM pg_stat_database;

-- Detailed conflict types (PostgreSQL 14+)
SELECT * FROM pg_stat_database_conflicts;
```

The Fix

Option 1 — `hot_standby_feedback = on` (the right fix for reporting standbys):

```
ini

# standby's postgresql.conf
hot_standby_feedback = on
```

This reports back to the primary which rows the standby is reading, preventing vacuum from removing them. Trade-off: can cause primary table bloat if standbys run very long queries.

Option 2 — Increase `max_standby_streaming_delay`:

```
ini
```

```
# standby's postgresql.conf
max_standby_streaming_delay = 30s    # default; raise for longer queries
max_standby_archive_delay = 30s
```

Option 3 — `old_snapshot_threshold` for acceptable staleness:

```
ini
```

```
# primary's postgresql.conf
old_snapshot_threshold = 60min
```

Takeaway

Set `hot_standby_feedback = on` for dedicated reporting standbys with long-running queries. Set a high `max_standby_streaming_delay` for standbys where occasional stale reads are acceptable. Monitor `pg_stat_database.conflicts` to track how often cancellations occur.

Issue 20: Connection Pooler Misconfiguration (PgBouncer)

The Problem

PgBouncer is almost universally deployed with PostgreSQL at scale — but misconfigured PgBouncer causes problems harder to debug than no pooler at all. Common traps include using session mode with long-lived connections (negating pooling benefits), `server_idle_timeout` too short (constant connection churn), `pool_mode = transaction` with `SET` commands (settings lost between transactions), and `max_client_conn` too low (application hits PgBouncer's own limit).

Diagnostic Queries

```
bash
```

```
# Connect to PgBouncer admin console
psql -p 6432 -U pgbouncer pgbouncer
```

```
SHOW POOLS;      -- cl_active, cl_waiting, sv_active, sv_idle
SHOW CLIENTS;    -- all client connections
SHOW SERVERS;    -- all server connections
SHOW CONFIG;     -- current configuration
```

The Fix

Choose the right pool mode:

Mode	Use when
session	Application uses SET, prepared statements, advisory locks
transaction	Stateless application; no session-level state
statement	Almost never; single-statement autocommit only

Production-recommended configuration:

```
ini

[databases]
mydb = host=pg-primary port=5432 dbname=mydb

[pgbouncer]
pool_mode = transaction
max_client_conn = 10000
default_pool_size = 25
min_pool_size = 5
reserve_pool_size = 5
reserve_pool_timeout = 3

server_idle_timeout = 600
server_lifetime = 3600
server_connect_timeout = 15

auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt

log_connections = 0
log_disconnections = 0
log_pooler_errors = 1
```

Handle SET commands in transaction mode:

```
ini
```

```
server_reset_query = DISCARD ALL
```

Takeaway

Use `transaction` mode unless your application uses session-level state. Set `max_client_conn` to a high value (5000–10000) so PgBouncer is never the bottleneck. Keep `default_pool_size` between 20–50 per database. Monitor `cl_waiting` in `SHOW POOLS` — any waiting clients mean your pool size is too small.

Issue 21: Out-of-Memory (OOM) Kills

The Problem

The Linux OOM killer terminates processes when the system runs out of memory. PostgreSQL processes are attractive targets because each backend process is large. An OOM kill of any PostgreSQL backend causes the entire cluster to restart, rolling back all in-flight transactions.

Symptoms: `dmesg` shows `Out of memory: Kill process (postgres)`, PostgreSQL restarts with no obvious reason in its own logs, all in-flight transactions are rolled back simultaneously.

Diagnostic Queries

```
bash

# Check kernel OOM log
dmesg | grep -i "out of memory\|oom"

# Check PostgreSQL log for unexpected restarts
grep "PANIC\|server process.*was terminated" /var/log/postgresql/*.log
```

The Fix

Calculate realistic memory ceiling:

```
Total PostgreSQL memory ≈
  shared_buffers
  + (max_connections × work_mem × avg_sort_ops_per_query)
  + (max_connections × 5MB per backend overhead)
```

```
+ (autovacuum_max_workers * maintenance_work_mem)
```

Leave at least 30–40% of RAM for the OS page cache.

Protect PostgreSQL from the OOM killer:

```
bash

# Lower OOM kill score – less likely to be targeted
echo -1000 > /proc/$(pgrep -x postgres | head -1)/oom_score_adj

# Or in the systemd unit file
# /etc/systemd/system/postgresql.service.d/oom.conf
[Service]
OOMScoreAdjust=-1000
```

Reduce per-session memory footprint:

```
sql

-- Low global work_mem; high values only for specific roles
ALTER ROLE reports_user SET work_mem = '256MB';
-- postgresql.conf: work_mem = 16MB
```

Enable huge pages to reduce per-process overhead:

```
ini

# postgresql.conf
huge_pages = on
```

Takeaway

OOM kills are a configuration math problem. Calculate your worst-case memory footprint before deploying. Set `OOMScoreAdjust = -1000` in the PostgreSQL systemd service to protect the process. Keep global `work_mem` low and use per-role overrides for heavy users. Never let total configured memory exceed 70% of physical RAM.

Issue 22: Disk Space Exhaustion

The Problem

PostgreSQL stops accepting writes when the data directory disk fills up. General disk exhaustion can come from log files accumulating in `pg_log`, temp files from spilling queries,

dead tuple bloat, core dumps from crashed backends, backup files left in the data directory, or WAL bloat from inactive replication slots.

Diagnostic Queries

```
sql

-- Database sizes
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
ORDER BY pg_database_size(datname) DESC;

-- Largest tables including indexes
SELECT
    schemaname || '.' || relname AS table_full_name,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||relname)) AS total_size,
    pg_size_pretty(pg_relation_size(schemaname||'.'||relname)) AS table_size,
    pg_size_pretty(pg_indexes_size(schemaname||'.'||relname)) AS indexes_size
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(schemaname||'.'||relname) DESC
LIMIT 20;

-- WAL and temp directories
SELECT 'WAL' AS dir, pg_size_pretty(sum(size)) FROM pg_ls_waldir()
UNION ALL
SELECT 'TEMP', pg_size_pretty(sum(size)) FROM pg_ls_tmpdir();
```

```
bash

# From OS level
du -sh /var/lib/postgresql/14/main/* | sort -h
find /var/lib/postgresql -name "core" -size +1M
```

The Fix

Immediate space recovery:

```
bash

# Compress old logs
find /var/lib/postgresql/14/main/pg_log -name "*.log" -mtime +7 -exec gzip {} \;

# Remove core dumps
find /var/lib/postgresql -name "core*" -delete
```

Prevent log accumulation:

```
ini

# postgresql.conf
log_rotation_age = 1d
log_rotation_size = 100MB
log_truncate_on_rotation = on
```

Use tablespaces to distribute data across disks:

```
sql

CREATE TABLESPACE large_data LOCATION '/mnt/data2/pg_tablespace';
ALTER TABLE large_log_table SET TABLESPACE large_data;
```

Takeaway

Alert at 80% disk usage. At 90%, begin immediate remediation. At 95%, you have minutes before PostgreSQL stops writing. Keep logs on a separate mount point from the data directory so log rotation issues cannot kill your database. Use

`pg_size_pretty(pg_database_size())` in your monitoring stack.

Issue 23: Temporary File Explosion

The Problem

When a sort or hash operation exceeds `work_mem`, PostgreSQL writes a temporary file to `base/pgsql_tmp/`. A single complex analytical query can generate gigabytes of temp files. If many such queries run concurrently, temp files can fill the data directory disk. Unlike WAL and bloat, temp files are deleted when the query finishes — but stale temp files can remain after crashes or disconnections.

Diagnostic Queries

```
sql

-- Total temp file stats per database
SELECT datname, temp_files, pg_size_pretty(temp_bytes)
FROM pg_stat_database
ORDER BY temp_bytes DESC;

-- Live temp files on disk right now
```

```
SELECT name, pg_size_pretty(size)
FROM pg_ls_tmpdir()
ORDER BY size DESC;

-- Enable logging for large temp files
-- postgresql.conf: log_temp_files = 104857600 (log files > 100MB)
```

In `EXPLAIN ANALYZE`, look for disk spills:

```
Sort Method: external merge  Disk: 42768kB    <-- bad: spilling to disk
Hash:  batches=8  Memory Usage: 4096kB        <-- bad: batched = spilled
```

The Fix

Set `temp_file_limit` to cap maximum temp space per session:

```
ini

# postgresql.conf
temp_file_limit = 10GB    # session exceeding this is cancelled
```

Route temp files to a separate disk:

```
sql

CREATE TABLESPACE fast_temp_disk LOCATION '/mnt/nvme/pg_temp';
```

```
ini

# postgresql.conf
temp_tablespaces = 'fast_temp_disk'
```

Clean stale temp files (only when PostgreSQL is stopped):

```
bash

# Only run when PostgreSQL is fully shut down
rm -f /var/lib/postgresql/14/main/base/pgsql_tmp/pgsql_tmp*
```

Takeaway

Set `log_temp_files = 100MB` to log any temp file larger than 100MB. Any query generating files that large is a candidate for `work_mem` tuning or query optimisation. Set `temp_file_limit` as a safety cap. Put temp files on a separate disk so a spill-heavy query cannot take out your entire data directory.

Issue 24: N+1 Query Patterns from ORMs

The Problem

N+1 is an ORM anti-pattern where loading a list of N objects triggers N additional queries to load related data. A page that should run 2 queries runs 2002 queries instead. Each query is individually fast, but the aggregate makes the request take seconds instead of milliseconds.

PostgreSQL shows nothing unusual in `pg_stat_activity` because each query completes quickly — but `pg_stat_statements` reveals the pattern.

Diagnostic Queries

```
sql

-- Find applications making many identical parameterised queries
SELECT
  left(query, 120) AS query_pattern,
  calls,
  round(mean_exec_time::numeric, 3) AS avg_ms,
  round(total_exec_time::numeric / 1000, 1) AS total_sec
FROM pg_stat_statements
WHERE calls > 1000
      AND mean_exec_time < 5    -- fast queries called an enormous number of times
ORDER BY calls DESC
LIMIT 20;
```

The Fix

Use a JOIN instead of separate queries:

```
sql

-- N+1 pattern (ORM-generated):
SELECT * FROM orders LIMIT 1000;
-- Then for each order:
SELECT * FROM customers WHERE id = $1; -- 1000 times!

-- Fix: single JOIN
SELECT o.*, c.name, c.email
FROM orders o
JOIN customers c ON c.id = o.customer_id
LIMIT 1000;
```

ORM-specific eager loading:

python

```
# Django
# BAD
orders = Order.objects.all()[:1000]
for o in orders:
    print(o.customer.name) # N queries

# GOOD
orders = Order.objects.select_related('customer').all()[:1000]
for o in orders:
    print(o.customer.name) # 1 JOIN query
```

javascript

```
// Sequelize
const orders = await Order.findAll({
  include: [{ model: Customer }],
  limit: 1000,
});
```

Use `IN` / `ANY` for batch lookups:

sql

```
-- Collect all IDs, then one query
SELECT * FROM customers
WHERE id = ANY(ARRAY[1,2,3,...,1000]);
```

Takeaway

N+1 is invisible to PostgreSQL monitoring — each query is fast. Use `pg_stat_statements` to find query patterns with enormous `calls` counts and low individual latency. Fix it at the ORM layer with eager loading. Set a per-request query count alarm in your APM tool; any request making more than 50 SQL queries is a red flag.

Issue 25: Partition Pruning Not Working

The Problem

Declarative table partitioning (PostgreSQL 10+) uses partition pruning to skip partitions that cannot contain relevant rows. When pruning does not work, a query scans all partitions, negating the performance benefit of partitioning entirely.

Pruning fails when the query filter is not directly on the partition key column, the filter uses a function on the partition key, there is a type mismatch, or `enable_partition_pruning` is off.

Diagnostic Queries

sql

```
-- Does the plan prune partitions?
EXPLAIN (ANALYZE)
SELECT * FROM events
WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31';
-- Look for: "Subplans Removed: N" – N is how many partitions were pruned
-- If missing: all partitions are being scanned

-- List all partitions and their ranges
SELECT
    relname,
    pg_get_expr(relpartbound, oid, true) AS partition_range
FROM pg_class
WHERE relispartition
    AND relname LIKE 'events_%'
ORDER BY relname;
```

The Fix

Use direct range comparisons on the partition key:

sql

```
-- BAD: function on partition key prevents pruning
SELECT * FROM events
WHERE date trunc('month', created_at) = '2024-01-01';
```

```
-- GOOD: direct range comparison
SELECT * FROM events
WHERE created_at >= '2024-01-01'
      AND created_at < '2024-02-01';
```

Ensure partition pruning is enabled:

```
sql

SHOW enable_partition_pruning; -- should be: on
SET enable_partition_pruning = on;
```

Match data types exactly:

```
sql

-- If partition key is timestamptz, filter must also be timestamptz
WHERE created_at >= '2024-01-01'::timestamptz
```

Enable partition-wise joins:

```
ini

# postgresql.conf
enable_partitionwise_join = on
enable_partitionwise_aggregate = on
```

Takeaway

Partition pruning only works when the filter is directly on the partition key column with compatible types and without wrapping functions. Always verify with `EXPLAIN` that `Subplans Removed` appears in the output. Partition by time using a `timestamptz` column and filter with range predicates — never use functions in the `WHERE` clause on the partition key.

Issue 26: Parallel Query Not Used

The Problem

PostgreSQL can execute certain queries using multiple parallel workers, dramatically reducing time for large sequential scans and aggregations. Parallel query is not used when the

table is too small to justify it, `max_parallel_workers_per_gather` is set to 0 or 1, the query contains a `LIMIT` that forces serial execution, or a function in the query path is marked `PARALLEL UNSAFE`.

Diagnostic Queries

```
sql

-- Check parallel query settings
SHOW max_parallel_workers_per_gather;
SHOW max_parallel_workers;
SHOW parallel_setup_cost;
SHOW parallel_tuple_cost;

-- Does this query use parallel workers?
EXPLAIN (ANALYZE)
SELECT count(*), category FROM large_events GROUP BY category;
-- Look for "Gather" or "Gather Merge" nodes
-- "Workers Planned: 4, Workers Launched: 4" = working correctly
-- "Workers Planned: 4, Workers Launched: 0" = workers failed to start
```

The Fix

Enable and tune parallel query:

```
ini

# postgresql.conf
max_parallel_workers_per_gather = 4      # workers per query node
max_parallel_workers = 8                 # total parallel workers
max_worker_processes = 16
parallel_setup_cost = 1000                # lower = more likely to parallelize
parallel_tuple_cost = 0.1
```

Set per-table parallel worker count:

```
sql

ALTER TABLE large_events SET (parallel_workers = 8);
```

Fix functions blocking parallel query

Fix functions blocking parallel query:

```
sql

-- Check a function's parallelism label
SELECT proname, proparallel FROM pg_proc WHERE proname = 'my_function';
-- proparallel: 's'=safe, 'r'=restricted, 'u'=unsafe

-- If the function is truly stateless and read-only:
ALTER FUNCTION my_function() PARALLEL SAFE;
```

Force parallel for a specific session:

```
sql

SET max_parallel_workers_per_gather = 8;
SET parallel_setup_cost = 0;
SELECT count(*) FROM large_events;
RESET max_parallel_workers_per_gather;
RESET parallel_setup_cost;
```

Takeaway

Parallel query gives free performance on analytical workloads with no schema changes required. Set `max_parallel_workers_per_gather = 4` as a baseline and increase for servers with many cores. Check `EXPLAIN ANALYZE` for "Workers Planned" vs "Workers Launched" — a mismatch means workers could not start. Review functions in hot query paths for `PARALLEL UNSAFE` labels.

Issue 27: Query Plan Regression After Upgrade

The Problem

PostgreSQL major version upgrades improve the query planner with new algorithms and cost model changes. While this almost always improves overall performance, specific queries may regress when the new planner makes different choices for a particular data distribution.

Signs: a previously fast query becomes slow after a major version upgrade, `EXPLAIN` output shows a different plan type (e.g., hash join → nested loop), or performance regression despite better hardware or settings.

Diagnostic Queries

```
sql
```

```
-- Check planner version-dependent settings
SELECT name, setting
FROM pg_settings
WHERE name IN (
    'enable_hashjoin', 'enable_mergejoin', 'enable_nestloop',
    'join_collapse_limit', 'geqo_threshold',
    'enable_parallel_hash', 'enable_memoize'
);

-- Compare actual vs estimated rows for the regressed query
EXPLAIN (ANALYZE, FORMAT JSON)
SELECT ... -- your slow query
```

The Fix

Step 1 — Update statistics first:

```
sql

VACUUM ANALYZE;
-- Re-check EXPLAIN — most regressions are statistics problems
```

Step 2 — Increase statistics target for regressed columns:

```
sql

ALTER TABLE orders ALTER COLUMN status SET STATISTICS 1000;
ANALYZE orders;
```

Step 3 — Use `pg_hint_plan` to pin the plan (targeted fix):

```
sql

CREATE EXTENSION IF NOT EXISTS pg_hint_plan;

/*+ HashJoin(orders customers) */
SELECT o.*, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id;
```

Step 4 — Disable a planner feature as a last resort:

```
sql

-- If hash join regressions are widespread across many queries
ALTER DATABASE mydb SET enable_hashjoin = off;
-- Address root cause before re-enabling
```

Takeaway

Plan regressions after upgrades are real but rare. Before disabling planner features globally, investigate whether the regression is a statistics problem — it usually is. Use `pg_hint_plan` as a temporary targeted fix for specific queries. Never run a major PostgreSQL upgrade without a staging environment that mirrors production query load and data distribution.

Issue 28: Index Not Used Due to Type Mismatch

The Problem

PostgreSQL is strict about types. If a query filters a column with a different type than the column's declared type, the planner may not be able to use the index — causing a silent sequential scan on a table with a perfectly good index.

Classic examples: column is `integer`, query passes a `varchar` like `WHERE id = '123'`; column is `timestampz`, query passes a `timestamp without time zone`; an ORM passes a 32-bit integer for a `bigint` column.

Diagnostic Queries

sql

```
-- Check column type
SELECT attname, atttypid::regtype AS data_type
FROM pg_attribute
WHERE attrelid = 'orders'::regclass AND attnum > 0;

-- Check indexes on the table
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
```



```
-- EXPLAIN reveals implicit casts
EXPLAIN SELECT * FROM orders WHERE id = '123';
-- "Filter: (id::text = '123')" → cast on the COLUMN side = index useless!
-- "Filter: (id = '123'::integer)" → cast on the VALUE side = index can be used
```

The Fix

Fix the application to pass the correct type:

python

```
# BAD – passes string for an integer column
cursor.execute("SELECT * FROM users WHERE id = %s", ("123",))

# GOOD – passes integer
cursor.execute("SELECT * FROM users WHERE id = %s", (123,))
```

java

```
// BAD
stmt.setString(1, "123"); // VARCHAR for INT column

// GOOD
stmt.setInt(1, 123);
```

Use explicit casts in the query as a fallback:

sql

```
SELECT * FROM orders WHERE id = '123'::integer;
```

Create a functional index if you cannot change the query:

sql

```
-- Index on the cast expression so the query can use it
CREATE INDEX idx_orders_id_text ON orders(id::text);
-- Now: WHERE id::text = '123' can use this index
```

ORM type safety (SQLAlchemy):

python

```
class Order(Base):
    id = Column(Integer, primary_key=True) # not String!
```

Takeaway

Type mismatches silently cause sequential scans on indexed columns. Always use `EXPLAIN` when adding an index to verify the planner actually uses it. When you see a cast on the column side in `EXPLAIN` output (e.g., `id::text`), the index cannot be used. Fix the root cause in the application layer; use functional indexes only as a last resort.

Issue 29: Slow COUNT(*) on Large Tables

The Problem

`SELECT COUNT(*) FROM large_table` requires a sequential scan in PostgreSQL because MVCC means different transactions see different row counts — there is no single global total rows number. On a table with 100 million rows, this can take 10+ seconds.

Diagnostic Queries

```
sql

-- How slow is it?
\timing
SELECT COUNT(*) FROM large_table;

-- Quick approximate count — usually within 0.1%
SELECT reltuples::bigint AS approx_count
FROM pg_class WHERE relname = 'large_table';

-- Slightly more accurate
SELECT n_live_tup AS approx_count
FROM pg_stat_user_tables WHERE relname = 'large_table';
```

The Fix

Option 1 — Use `reltuples` for approximate counts (instant):

```
sql

SELECT reltuples::bigint AS approx_count
FROM pg_class WHERE relname = 'events';
-- Updated by ANALYZE; within ~1% for well-vacuumed tables
```

Option 2 — Index-only scan for filtered counts:

```
sql
-- Requires an index on (status)
-- Uses Index Only Scan, not a full sequential scan
SELECT COUNT(*) FROM orders WHERE status = 'pending';
```

Option 3 — Counter table maintained by triggers:

```
sql

CREATE TABLE table_counts (
  table_name text PRIMARY KEY,
  cnt bigint DEFAULT 0
);
INSERT INTO table_counts VALUES ('orders', 0);

CREATE OR REPLACE FUNCTION update_order_count() RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    UPDATE table_counts SET cnt = cnt + 1 WHERE table_name = 'orders';
  ELSIF TG_OP = 'DELETE' THEN
    UPDATE table_counts SET cnt = cnt - 1 WHERE table_name = 'orders';
  END IF;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER orders_count_trigger
  AFTER INSERT OR DELETE ON orders
  FOR EACH ROW EXECUTE FUNCTION update_order_count();
```

Option 4 — Parallel COUNT (exact but faster):

```
sql

SET max_parallel_workers_per_gather = 8;
SET parallel_setup_cost = 0;
SELECT COUNT(*) FROM large_table;
```

Takeaway

Exact `COUNT(*)` on large tables is expensive by design in MVCC databases. For dashboards and UIs, `reltuples` or `n_live_tup` approximations are accurate enough (within 1%) and return instantly. Reserve exact counts for business-critical operations. If you need exact real-time counts, maintain a counter table with triggers.

Issue 30: LIKE Queries Ignoring Indexes

The Problem

B-tree indexes support `LIKE 'prefix%'` (prefix match) but not `LIKE '%suffix'` (suffix match) or `LIKE '%infix%'` (contains). The standard text operator class is needed for prefix LIKE on a B-tree index in non-C locales, and a `pg_trgm` trigram index is needed for arbitrary substring searches.

Diagnostic Queries

```
sql

-- Does this LIKE query use an index?
EXPLAIN SELECT * FROM products WHERE name LIKE 'apple%';
-- "Seq Scan" = no index; "Index Scan" = good

-- What indexes exist on this column?
SELECT indexname, indexdef FROM pg_indexes
WHERE tablename = 'products' AND indexdef ILIKE '%name%';
```

The Fix

Prefix LIKE — use `text_pattern_ops`:

```
sql

-- Standard index does NOT reliably work for LIKE in non-C locales
-- This one does:
CREATE INDEX idx_products_name_pattern
  ON products(name text_pattern_ops);

SELECT * FROM products WHERE name LIKE 'apple%'; -- uses index
```

Substring LIKE — use `pg_trgm` with GIN index:

```
sql

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- GIN trigram index: supports LIKE '%text%' and ILIKE
CREATE INDEX idx_products_name_trgm
  ON products USING gin(name gin_trgm_ops);

SELECT * FROM products WHERE name LIKE '%apple%'; -- uses index
SELECT * FROM products WHERE name ILIKE '%Apple%'; -- case-insensitive, uses index
SELECT * FROM products WHERE name % 'apple%'; -- similarity search
```

```
SELECT * FROM products WHERE name % 'apple'; -- Similarity Search
```

Case-insensitive exact match — functional index on `lower()`:

```
sql

CREATE INDEX idx_products_name_lower ON products(lower(name));
SELECT * FROM products WHERE lower(name) = lower('Apple');
```

Or use the `citext` extension:

```
sql

CREATE EXTENSION citext;
ALTER TABLE products ALTER COLUMN name TYPE citext;
SELECT * FROM products WHERE name = 'Apple'; -- case-insensitive automatically
```

Takeaway

B-tree indexes do not help with `LIKE '%substring%'`. Install `pg_trgm` and create a GIN index for any column where substring searches are common. For prefix-only `LIKE 'prefix%'`, create the index with `text_pattern_ops`. For case-insensitive exact matches, use a functional index on `lower(column)` or the `citext` type.

Issue 31: JSON/JSONB Query Performance

The Problem

`JSONB` is powerful but querying deeply nested JSON without proper indexing causes sequential scans and slow extraction. Common performance traps include querying on a JSON key without an index, using `->` vs `->>` incorrectly in filters, storing data in JSONB that should be in normalized columns, and missing GIN indexes.

Diagnostic Queries

```
sql

-- Is the JSONB query using an index?
EXPLAIN (ANALYZE)
SELECT * FROM events WHERE data->>'event_type' = 'click';
-- "Seq Scan" = no index; "Bitmap Index Scan" = good

-- Existing indexes on JSONB column
SELECT indexname, indexdef FROM pg_indexes
```

```
SELECT indexname, indexdef FROM pg_indexes  
WHERE tablename = 'events' AND indexdef LIKE '%data%';
```

The Fix

GIN index for containment and key existence queries:

```
sql  
  
-- Full GIN index: supports @>, ?, ?|, ?& operators  
CREATE INDEX idx_events_data_gin ON events USING gin(data);  
  
SELECT * FROM events WHERE data @> '{"event_type": "click"}'; -- uses index  
SELECT * FROM events WHERE data ? 'user_id'; -- uses index
```

Functional index for specific key access:

```
sql  
  
-- More selective than a full GIN; smaller index  
CREATE INDEX idx_events_event_type ON events((data->>'event_type'));  
  
SELECT * FROM events WHERE data->>'event_type' = 'click'; -- uses index
```

JSONB path index (PostgreSQL 12+):

```
sql  
  
-- jsonb_path_ops: smaller than default GIN, faster for @> and @?  
CREATE INDEX idx_events_data_path  
ON events USING gin(data jsonb_path_ops);  
  
SELECT * FROM events WHERE data @? '$.user_id ? (@ > 1000)';
```

Extract frequently queried fields to generated columns:

```
sql  
  
ALTER TABLE events ADD COLUMN user_id bigint  
GENERATED ALWAYS AS ((data->>'user_id')::bigint) STORED;  
  
CREATE INDEX idx_events_user_id ON events(user_id);  
-- Now: WHERE user_id = 42 uses a clean btree index
```

Takeaway

Use `jsonb_path_ops` GIN index for `@>` and `@?` operator queries — it is smaller and faster than the default GIN. Use functional indexes on specific frequently-queried keys. If

you always filter on the same two or three JSONB fields, extract them to generated columns and index those instead. Never rely on JSONB to avoid schema design.

Issue 32: Full-Text Search Performance

The Problem

PostgreSQL has built-in full-text search (`tsvector`, `tsquery`) that is fast when properly indexed, but brutally slow when not. Common mistakes include computing `tsvector` on every query row instead of storing it in a column, missing GIN indexes on the `tsvector` column, using `LIKE '%keyword%'` instead of the `@@` operator, and not setting the right text search configuration (language).

Diagnostic Queries

```
sql

-- Is the full-text query using an index?
EXPLAIN (ANALYZE)
SELECT * FROM articles
WHERE to_tsvector('english', body) @@ plainto_tsquery('english', 'postgres perfor
-- "Seq Scan" = no index (computing tsvector per row – very slow)
-- "Bitmap Index Scan" = good
```

The Fix

Step 1 — Store the `tsvector` as a generated column:

```
sql

ALTER TABLE articles
ADD COLUMN search_vector tsvector
GENERATED ALWAYS AS (
    to_tsvector('english',
        coalesce(title, '') || ' ' || coalesce(body, ''))
) STORED;
```

Step 2 — Create a GIN index on the stored vector:

```
sql

CREATE INDEX idx_articles_search ON articles USING gin(search_vector);
```

Step 3 — Query using the stored column:

```
sql

SELECT id, title,
       ts_rank(search_vector, query) AS relevance_rank
FROM articles,
     plainto_tsquery('english', 'postgres performance') query
WHERE search_vector @@ query
ORDER BY relevance_rank DESC
LIMIT 20;
```

Add highlighted excerpts:

```
sql

SELECT
  id, title,
  ts_headline('english', body,
    plainto_tsquery('english', 'postgres performance'),
    'MaxWords=30, MinWords=15'
  ) AS excerpt
FROM articles
WHERE search_vector @@ plainto_tsquery('english', 'postgres performance')
ORDER BY ts_rank(search_vector,
  plainto_tsquery('english', 'postgres performance')) DESC
LIMIT 10;
```

Takeaway

Store `tsvector` as a `GENERATED ALWAYS AS ... STORED` column and index it with GIN. Never compute `to_tsvector()` inline in a WHERE clause on a large table — it scans and recomputes for every row. Use `plainto_tsquery` for user input, and `websearch_to_tsquery` (PostgreSQL 11+) for Google-style query syntax with operators like `OR` and `-exclude`.

Issue 33: Sequence Exhaustion

The Problem

PostgreSQL sequences for primary keys (`SERIAL`, `BIGSERIAL`, identity columns) have a maximum value. When the maximum is reached, inserts fail with:


```
ERROR: nextval: reached maximum value of sequence "users_id_seq" (2147483647)
```

`SERIAL` uses `integer` (max ~2.1 billion). In high-volume systems, this is reachable.

`BIGSERIAL` uses `bigint` (max ~9.2 quintillion), which is effectively unlimited for any sane use case.

Diagnostic Queries

```
sql

-- Check all sequences: current value, max, and percentage used
SELECT
  n.nspname AS schema,
  s.relname AS sequence_name,
  format_type(t.sectypid, null) AS data_type,
  t.seqmax AS max_value,
  last_value,
  t.seqmax - last_value AS remaining,
  round(100.0 * last_value / t.seqmax, 2) AS pct_used
FROM pg_class s
JOIN pg_namespace n ON n.oid = s.relnamespace
JOIN pg_sequence t ON t.seqrelid = s.oid
WHERE s.relkind = 'S'
ORDER BY pct_used DESC NULLS LAST;
```

The Fix

Convert `SERIAL` (integer) to `BIGSERIAL` (bigint):

```
sql

ALTER SEQUENCE users_id_seq AS bigint;
ALTER TABLE users ALTER COLUMN id TYPE bigint;
```

Use identity columns (PostgreSQL 10+, preferred):

```
sql

-- New table
CREATE TABLE users (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email text NOT NULL
);

-- Convert existing
ALTER TABLE users ALTER COLUMN id
  ADD GENERATED ALWAYS AS IDENTITY
```

```
ADD GENERATED ALWAYS AS IDENTITY;
```

Use UUIDs to eliminate sequence exhaustion entirely:

```
sql

CREATE TABLE events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(), -- PG 13+, no extension needed
  created_at timestamptz DEFAULT now()
);
```

Alert before exhaustion:

```
sql

-- Alert if any integer sequence is more than 75% used
SELECT sequence_name,
       round(100.0 * last_value / seqmax, 2) AS pct_used
FROM pg_class
JOIN pg_sequence ON seqrelid = oid
WHERE relkind = 'S'
      AND seqtypid = 'integer'::regtype::oid
      AND round(100.0 * last_value / seqmax, 2) > 75;
```

Takeaway

Default `SERIAL` uses a 32-bit integer — good for 2 billion rows. Always use `BIGSERIAL` or `bigint GENERATED AS IDENTITY` for any table that could grow large. Run the sequence exhaustion query monthly on production. If you find a sequence over 80% used, migrate to `bigint` immediately — you cannot add values once the sequence is exhausted.

Issue 34: Constraint Violations at Scale

The Problem

As data volume grows, constraint violations surface in ways that were never an issue at small scale: unique constraint violations from concurrent inserts (race conditions), foreign key constraint violations during bulk loads, and check constraint failures from application logic bugs. These cause transaction rollbacks, application errors, and in bad cases, data inconsistency.

Diagnostic Queries

```
sql

-- Find all constraints on a table
```

```
-- Find all constraints on a table
SELECT
    conname AS constraint_name,
    contype AS type, -- p=primary, u=unique, f=foreign, c=check
    pg_get_constraintdef(oid) AS definition
FROM pg_constraint
WHERE conrelid = 'orders'::regclass;

-- Check for deferrable constraints
SELECT conname, condeferrable, condeferred
FROM pg_constraint
WHERE conrelid = 'orders'::regclass;
```

The Fix

Unique constraint violations — use `INSERT ... ON CONFLICT`:

```
sql

-- Upsert: atomic, no race conditions
INSERT INTO users (email, name) VALUES ('a@b.com', 'Alice')
ON CONFLICT (email) DO UPDATE
    SET name = EXCLUDED.name,
        updated_at = now();

-- Skip if already exists
INSERT INTO events (id, payload) VALUES (gen_random_uuid(), '{}')
ON CONFLICT DO NOTHING;
```

Bulk load with foreign key deferral:

```
sql

BEGIN;
SET CONSTRAINTS ALL DEFERRED;

COPY orders FROM '/data/orders.csv' WITH (FORMAT csv);
COPY order_items FROM '/data/items.csv' WITH (FORMAT csv);
-- FK checks run at commit, not mid-transaction

COMMIT;
```

Make constraints deferrable for complex applications:

```
sql

ALTER TABLE order_items
```

```
ADD CONSTRAINT fk_order FOREIGN KEY (order_id)
REFERENCES orders(id)
DEFERRABLE INITIALLY DEFERRED;
```

Takeaway

Use `INSERT ... ON CONFLICT` instead of application-level "check then insert" patterns — it is atomic and race-condition-free. For bulk loads, defer FK constraints, then verify integrity after the load completes. Make performance-critical FK constraints deferrable so complex multi-table inserts can succeed within a single transaction.

Issue 35: Foreign Key Locking Overhead

The Problem

Foreign key constraints cause implicit lock acquisitions on every write: on `INSERT` into the child table, PostgreSQL acquires a `ShareLock` on the referenced parent row; on `DELETE` from the parent table, PostgreSQL locks to check no child rows exist. In high-concurrency systems, this causes contention that is invisible in slow query logs — queries are fast individually, but they queue behind FK lock waits.

Diagnostic Queries

sql

```
-- Missing indexes on FK columns (the most common cause of FK overhead)
```

```
SELECT
  tc.table_name AS child_table,
  kcu.column_name AS fk_column,
  ccu.table_name AS parent_table
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  USING (constraint_name, table_schema)
JOIN information_schema.referential_constraints rc
  USING (constraint_name, table_schema)
JOIN information_schema.constraint_column_usage ccu
  USING (constraint_name, table_schema)
WHERE NOT EXISTS (
```

```

SELECT 1 FROM pg_index i
JOIN pg_class c ON c.oid = i.indrelid
JOIN pg_attribute a ON a.attrelid = c.oid
    AND a.attnum = ANY(i.indkey)
WHERE c.relname = tc.table_name
    AND a.attname = kcu.column_name
);

```

The Fix

Index every foreign key column on the child table (most important fix):

```

sql

-- PostgreSQL does NOT create these automatically
CREATE INDEX CONCURRENTLY idx_order_items_order_id
    ON order_items(order_id);

CREATE INDEX CONCURRENTLY idx_order_items_product_id
    ON order_items(product_id);

```

Without this index, every **DELETE** from the parent table does a full sequential scan of the child table to find dependent rows.

Batch deletes on parent rows to reduce lock contention:

```

sql


DO $$
DECLARE deleted int;
BEGIN
    LOOP
        DELETE FROM orders
        WHERE id IN (
            SELECT id FROM orders
            WHERE status = 'archived'

```

```

WHERE status = 'archived'
AND created_at < now() - interval '2 years'
LIMIT 1000
);
GET DIAGNOSTICS deleted = ROW_COUNT;
EXIT WHEN deleted = 0;
PERFORM pg_sleep(0.1); -- brief pause between batches
END LOOP;
END $$;

```

Takeaway

Always index foreign key columns on the referencing (child) table. PostgreSQL does not create these indexes automatically — it is one of the most commonly missed optimisations in schema design. An unindexed FK column causes a full sequential scan of the child table on every DELETE from the parent. Run a query to check all FK columns for missing indexes as part of your regular schema review.

Issue 36: Long-Running Schema Migrations

The Problem

Schema changes (`ALTER TABLE`, `CREATE INDEX`, `DROP COLUMN`) acquire locks that block application queries. A migration that takes 10 minutes holds a lock for 10 minutes — during which every query against that table queues up, eventually exhausting connections. Additionally, if a migration fails mid-way, it may leave the schema in an inconsistent state that blocks retries.

Diagnostic Queries

```

sql

-- Is a migration currently blocking everything?
SELECT
  pid,
  now() - query_start AS duration,
  state,
  wait_event_type,
  left(query, 120) AS query
FROM pg_stat_activity
WHERE query LIKE 'ALTER TABLE%'
      OR query LIKE 'CREATE INDEX%'
      OR query LIKE 'REINDEX%'
ORDER BY duration DESC;

```

The Fix

Use `CREATE INDEX CONCURRENTLY` for new indexes:

```
sql

-- Blocks writes for the duration (minutes on large tables):
CREATE INDEX idx_orders_status ON orders(status);

-- Does NOT block reads or writes:
CREATE INDEX CONCURRENTLY idx_orders_status ON orders(status);
```

Add columns safely on older PostgreSQL (≤ 10):

```
sql

-- Step 1: nullable column, no default – instant
ALTER TABLE orders ADD COLUMN priority integer;

-- Step 2: backfill in batches
UPDATE orders SET priority = 0
WHERE priority IS NULL AND id BETWEEN 1 AND 100000;
-- ... repeat for all id ranges

-- Step 3: add constraint after backfill
ALTER TABLE orders ALTER COLUMN priority SET DEFAULT 0;
ALTER TABLE orders ALTER COLUMN priority SET NOT NULL;
```

Set `lock_timeout` to fail fast rather than queue:

```
sql

SET lock_timeout = '2s';
ALTER TABLE orders ADD COLUMN ...;
-- Fails cleanly if it cannot get the lock; retry during low-traffic window
```

Zero-downtime column rename (expand-contract pattern):

```
sql
```

```
-- Step 1: Add new column
ALTER TABLE users ADD COLUMN email_address text;
-- Step 2: Deploy app to write to both columns
-- Step 3: Backfill new column
UPDATE users SET email_address = email WHERE email_address IS NULL;
-- Step 4: Deploy app to read from new column
-- Step 5: Drop old column
ALTER TABLE users DROP COLUMN email;
```

Takeaway

Always use `CREATE INDEX CONCURRENTLY` for production indexes. Set `lock_timeout = '2s'` in migration scripts to fail fast rather than queue. Use the expand-contract pattern for column renames and non-additive changes. Test migrations on a production-sized data copy — a migration that takes 30 seconds on staging may take 30 minutes on production.

[issue-37](#)

Issue 37: Table-Level Lock from ALTER TABLE

The Problem

Many `ALTER TABLE` commands acquire `AccessExclusiveLock`, blocking all reads and writes. Some also rewrite the entire table, making them $O(n)$ operations.

Commands that cause table rewrites (slow):

- `ADD COLUMN ... NOT NULL DEFAULT expr` (PostgreSQL 10 and earlier)
- `ALTER COLUMN ... TYPE` (most type changes)
- `SET TABLESPACE`

Commands that do NOT rewrite the table (fast):

- `ADD COLUMN` with nullable, no default (instant)
- `ADD COLUMN` with default (PostgreSQL 11+, instant)
- `ADD CONSTRAINT ... NOT VALID` (instant validation, then separate validation step)
- `VALIDATE CONSTRAINT` (ShareUpdateExclusiveLock — allows concurrent reads/writes)

The Fix

Add constraints without blocking writes:


```

sql

-- Step 1: instant lock, then released
ALTER TABLE orders
  ADD CONSTRAINT chk_amount_positive CHECK (amount > 0) NOT VALID;

-- Step 2: validate without blocking writes
ALTER TABLE orders VALIDATE CONSTRAINT chk_amount_positive;
-- ShareUpdateExclusiveLock – reads and writes continue

```

Change column type without table rewrite:

```

sql

-- Add new column
ALTER TABLE orders ADD COLUMN amount_v2 numeric(12,2);

-- Backfill in batches
UPDATE orders SET amount_v2 = amount::numeric(12,2)
WHERE id BETWEEN 1 AND 100000;
-- ... continue for all ranges

-- Swap column names
ALTER TABLE orders
  RENAME COLUMN amount TO amount_old,
  RENAME COLUMN amount_v2 TO amount;

-- Drop old after verifying correctness
ALTER TABLE orders DROP COLUMN amount_old;

```

Takeaway

Know which `ALTER TABLE` operations rewrite the table and which do not. Use `ADD CONSTRAINT ... NOT VALID` + `VALIDATE CONSTRAINT` as a two-step process for adding check and FK constraints without extended blocking. In PostgreSQL 11+, adding columns with defaults is instant. For type changes, use the add-backfill-swap-drop pattern.

Issue 38: Connection Storms After Restart

The Problem

When PostgreSQL restarts (after a crash, maintenance, or upgrade), all application connection pools try to reconnect simultaneously. This connection storm overwhelms PostgreSQL's connection setup overhead, causes authentication log spam, and leads to cascading timeouts. If the restart was due to a crash, WAL recovery may take time, causing application retries to amplify the load further.

The Fix

Configure connection retry with exponential backoff and jitter:

```
python

import psycopg2, time, random

def connect_with_backoff(dsn, max_retries=10):
    for attempt in range(max_retries):
        try:
            return psycopg2.connect(dsn)
        except psycopg2.OperationalError:
            if attempt == max_retries - 1:
                raise
            sleep = min((2 ** attempt) + random.uniform(0, 1), 60)
            time.sleep(sleep)
```

Use PgBouncer as a connection buffer:

PgBouncer queues client connections while PostgreSQL is restarting, so clients queue in PgBouncer rather than hammering PostgreSQL with simultaneous connection attempts.

```
ini

# pgbouncer.ini
query_wait_timeout = 30          # clients wait up to 30s for server
server_connect_timeout = 15
server_login_retry = 15
```

Reserve superuser connections for DBA access during recovery:

```
ini

# postgresql.conf
superuser_reserved_connections = 10
```

Takeaway

Connection storms are a distributed systems problem — all clients retry simultaneously. Implement exponential backoff with jitter in your connection retry logic. PgBouncer buffers reconnect storms. Set `server_connect_timeout` and `query_wait_timeout` in PgBouncer so queued clients fail gracefully if recovery takes longer than expected.

[issue-39](#)

Issue 39: Logical Replication Lag and Conflicts

The Problem

Logical replication (PostgreSQL 10+) replicates changes at the row level, allowing replication between different major versions and partial table replication. But it has unique failure modes: conflicts on unique constraints (subscriber has a row the publisher tries to INSERT), replication lag when the subscriber cannot keep up, worker process crashes, and failures when the publisher adds a column not on the subscriber.

Diagnostic Queries

```
sql

-- On publisher: replication slot for logical replication
SELECT
    slot_name,
    active,
    confirmed_flush_lsn,
    pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS bytes_lag
FROM pg_replication_slots
WHERE slot_type = 'logical';

-- On subscriber: worker status
SELECT
    subname,
    pid,
    received_lsn,
    latest_end_lsn,
    latest_end_time
FROM pg_stat_subscription;
```

The Fix

Resolve unique key conflicts on the subscriber:

```
sql

-- Delete the conflicting row on the subscriber; let replication replay it
```

```
DELETE FROM orders WHERE id = 12345;
```

```
-- Or refresh the whole table subscription  
ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION;
```

Re-enable a stopped subscription:

```
sql  
  
ALTER SUBSCRIPTION my_sub ENABLE;  
  
-- Or recreate from scratch for a clean start  
DROP SUBSCRIPTION my_sub;  
CREATE SUBSCRIPTION my_sub  
  CONNECTION 'host=primary dbname=mydb'  
  PUBLICATION my_pub;
```

Isolate subscriber from publisher schema changes:

```
sql  
  
-- Publish only specific columns to prevent subscriber failures on new columns  
ALTER PUBLICATION my_pub SET TABLE orders (id, status, amount, updated_at);
```

Speed up logical replication on the subscriber:

```
ini  
  
# subscriber's postgresql.conf  
max_logical_replication_workers = 8  
max_sync_workers_per_subscription = 4
```

Takeaway

Logical replication conflicts require manual intervention — PostgreSQL cannot auto-resolve them. Monitor `pg_stat_subscription` and `pg_replication_slots` for lag. Define column lists in publications to isolate the subscriber from publisher schema changes. Enable `subdisableonerr = false` (default) so the subscription stops on conflict rather than silently skipping rows.

Issue 40: Backup Failure from pg_dump

The Problem

`pg_dump` is the most common backup tool for PostgreSQL, but it fails or produces unusable backups when the database is too large for the available destination disk, a long-running transaction prevents a consistent snapshot, network interruption breaks a remote dump, version mismatch between client and server, or the dump is taken from a hot standby with certain limitations.

The Fix

Use custom format for compression and parallelism:

```
bash

# Custom format: compressed, supports parallel restore
pg_dump -Fc -j 4 -h localhost -U postgres -d mydb \
  -f /backup/mydb_$(date +%Y%m%d).dump

# Directory format: supports parallel dump and restore
pg_dump -Fd -j 8 -h localhost -U postgres -d mydb \
  -f /backup/mydb_$(date +%Y%m%d)/

pg_restore -Fd -j 8 -d mydb_restore /backup/mydb_20240101/
```

Stream directly to S3 to avoid local disk constraints:

```
bash

pg_dump -Fc -h localhost -U postgres mydb | \
  aws s3 cp - s3://my-backup-bucket/mydb_$(date +%Y%m%d).dump
```

Use `pg_basebackup` for large databases:

```
bash

# Physical backup – much faster than pg_dump for large databases
pg_basebackup -h localhost -U replication_user \
  -D /backup/basebackup_$(date +%Y%m%d) \
  -Ft -z -P --wal-method=stream
```

Always verify backup integrity:

```
bash

# Test restore to a separate instance
createdb mydb_test
pg_restore -Fc -d mydb_test /backup/mydb_20240101.dump
psql -d mydb_test -c "SELECT count(*) FROM orders;"
```

Takeaway

Always use custom format (`-Fc`) or directory format (`-Fd`) — never plain SQL for production databases. Stream large dumps directly to S3 to avoid local disk constraints. Test restores regularly — an untested backup is not a backup. For databases over 100GB, prefer `pg_basebackup` with WAL archiving over `pg_dump`.

[issue-41](#)

Issue 41: Point-in-Time Recovery Misconfiguration

The Problem

Point-in-time recovery (PITR) allows restoring a database to any moment in time by replaying WAL from a base backup. Misconfiguration causes PITR to fail when you need it most:

`archive_mode = off` (WAL not being archived), `archive_command` failing silently, WAL archives accumulating but never being tested, or base backups too old to meet RTO.

Diagnostic Queries

```
sql

-- Check archiving is enabled and working
SHOW archive_mode;
SHOW archive_command;
SHOW wal_level;  -- must be 'replica' or 'logical' for archiving

-- Archive status and failure count
SELECT
    archived_count,
    last_archived_wal,
    last_archived_time,
    failed_count,
    last_failed_wal,
    last_failed_time
FROM pg_stat_archiver;
```

The Fix

Enable WAL archiving with WAL-G (recommended for production):

```
ini

# postgresql.conf
wal_level = replica
```

```
archive_mode = on
archive_command = 'wal-g wal-push %p'
```

```
bash

# Environment variables for WAL-G
export WALG_S3_PREFIX=s3://my-wal-bucket/mydb
export AWS_REGION=us-east-1

# Take a base backup
wal-g backup-push /var/lib/postgresql/14/main
```

Test PITR recovery regularly (monthly minimum):

```
bash

# Restore to a specific point in time
wal-g backup-fetch /var/lib/postgresql/14/restore LATEST

# recovery.signal (PostgreSQL 12+)
cat > /var/lib/postgresql/14/restore/recovery.signal << EOF
EOF

# postgresql.conf on restore instance
restore_command = 'wal-g wal-fetch %f %p'
recovery_target_time = '2024-01-15 14:30:00'
recovery_target_action = 'promote'
```

Verify `pg_stat_archiver.failed_count = 0` daily in monitoring.

Takeaway

PITR is only as good as your most recent tested restore. Archive WAL continuously using WAL-G or Barman. Verify `pg_stat_archiver.failed_count = 0` daily. Run a full PITR restore test monthly. Know your RTO: a 100GB base backup plus 8 hours of WAL replay might take 45 minutes — verify this against your recovery time objective.

Issue 42: SSL/TLS Connection Issues

The Problem

PostgreSQL supports SSL/TLS for encrypted connections but configuration errors are common: expired SSL certificates causing connection failures, hostname mismatches in certificate causing verification failures, applications using `sslmode=disable` when the

certificate causing verification failures, applications using `sslmode=disable` when the server requires SSL, and performance overhead from SSL on very high connection-rate systems.

Diagnostic Queries

```
sql

-- Is SSL enabled?
SHOW ssl;

-- Current connection's SSL status
SELECT ssl, cipher, bits FROM pg_stat_ssl
WHERE pid = pg_backend_pid();

-- All connections and encryption status
SELECT sa.pid, sa.username, sa.application_name, ss.ssl, ss.cipher
FROM pg_stat_activity sa
LEFT JOIN pg_stat_ssl ss ON sa.pid = ss.pid
WHERE sa.pid != pg_backend_pid()
ORDER BY ss.ssl;
```

```
bash

# Check certificate expiry
openssl x509 -in /var/lib/postgresql/14/main/server.crt -noout -dates
```

The Fix

Enable and configure SSL:

```
ini

# postgresql.conf
ssl = on
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
ssl_ca_file = 'root.crt'
ssl_min_protocol_version = 'TLSv1.2'
```

Require SSL for remote connections in pg_hba.conf:

```
# pg_hba.conf
local    all  all                                peer
hostssl  all  all  0.0.0.0/0                    scram-sha-256
```

Application SSL configuration (verify-full = highest security):


```
python
```

```
conn = psycopg2.connect(  
    host="mydb.example.com",  
    dbname="mydb",  
    user="myuser",  
    sslmode="verify-full",          # not just "require"  
    sslrootcert="/etc/ssl/certs/root.crt"  
)
```

Automate certificate renewal:

```
bash
```

```
# Certbot deploy hook for Let's Encrypt  
cp /etc/letsencrypt/live/mydb.example.com/fullchain.pem \  
  /var/lib/postgresql/14/main/server.crt  
cp /etc/letsencrypt/live/mydb.example.com/privkey.pem \  
  /var/lib/postgresql/14/main/server.key  
chown postgres:postgres /var/lib/postgresql/14/main/server.{crt,key}  
chmod 600 /var/lib/postgresql/14/main/server.key  
su -c "pg_ctl reload -D /var/lib/postgresql/14/main" postgres
```

Takeaway

Use `sslmode=verify-full` in application connection strings — not `require` or `disable`. `verify-full` checks both encryption and hostname, preventing man-in-the-middle attacks. Set calendar reminders for SSL certificate expiry, or automate with Let's Encrypt and certbot. Monitor `pg_stat_ssl` to ensure all non-local connections are encrypted.

Issue 43: Extension Conflicts and Version Mismatches

The Problem

PostgreSQL extensions can conflict with each other, with the PostgreSQL version, or with themselves across schema changes. Common failures include extension versions not available in the current PostgreSQL, extensions installed in the wrong schema, `CREATE EXTENSION` failing because old functions still exist, extension upgrades leaving stale objects, and `shared_preload_libraries` referencing a missing extension.

Diagnostic Queries

```

sql

-- All installed extensions with versions
SELECT extname, extversion, extschema::regnamespace AS schema
FROM pg_extension
ORDER BY extname;

-- Extensions with available updates
SELECT name, installed_version, default_version
FROM pg_available_extensions
WHERE installed_version IS NOT NULL
      AND installed_version != default_version;

-- Verify shared_preload_libraries
SHOW shared_preload_libraries;

```

The Fix

Install extensions in the correct schema:

```

sql

CREATE EXTENSION pg_trgm SCHEMA public;
CREATE EXTENSION postgis SCHEMA public;

```

Upgrade extensions after a major PostgreSQL version upgrade:

```

sql

ALTER EXTENSION postgis UPDATE;
ALTER EXTENSION pg_stat_statements UPDATE;

-- Check target version first
SELECT * FROM pg_available_extension_versions WHERE name = 'postgis';

```

Fix a broken extension installation:

```

sql

-- If CREATE EXTENSION fails because objects already exist:
DROP EXTENSION IF EXISTS pg_trgm CASCADE;
CREATE EXTENSION pg_trgm;

```

Validate `(shared_preload_libraries)` before restarting:

```
bash

# Check the extension .so file exists on disk
ls /usr/lib/postgresql/14/lib/pg_stat_statements.so

# Validate config without restarting
postgres --config-file=/etc/postgresql/14/main/postgresql.conf \
-C shared_preload_libraries 2>&1
```

Takeaway

After every major PostgreSQL version upgrade, run `ALTER EXTENSION <name> UPDATE` for all installed extensions. Before adding to `shared_preload_libraries`, verify the extension is installed and its `.so` file exists on disk. Keep a registry of all installed extensions and their required PostgreSQL version ranges — essential for upgrade planning.

Issue 44: Corrupt Indexes and Data Pages

The Problem

Data corruption in PostgreSQL is rare but real. It can be caused by storage hardware failure, memory errors (ECC RAM failure), kernel or file system bugs, or improper shutdown (rare — WAL protects against most of this).

Symptoms include `ERROR: invalid page in block N`, `ERROR: could not read block N: read only 0 of 8192 bytes`, queries returning unexpected NULL values, or `pg_dump` failing on specific tables.

Diagnostic Queries

```
sql

-- Check for index corruption (requires amcheck extension)
CREATE EXTENSION IF NOT EXISTS amcheck;

-- Quick check
SELECT bt_index_check('my_table_pkey');

-- Thorough check including heap verification
SELECT bt_index_parent_check('my_table_pkey', true);

-- Check a data page (requires pageinspect)
```

```
check a data page (requires pageinspect);  
CREATE EXTENSION IF NOT EXISTS pageinspect;  
SELECT * FROM page_header(get_raw_page('my_table', 0));
```

bash

```
# Check data checksums on a running server (PostgreSQL 12+)  
pg_checksums -D /var/lib/postgresql/14/main --check  
  
# Filesystem check (requires server to be stopped)  
fsck /dev/sdb1
```

The Fix

Rebuild a corrupt index (safest first step):

sql

```
REINDEX INDEX CONCURRENTLY my_corrupt_index;  
REINDEX TABLE CONCURRENTLY my_table;  
REINDEX DATABASE mydb;
```

Recover data from a corrupt page (data loss risk — use only if no backup):

sql

```
-- Skip corrupt pages; fills them with zeros instead of erroring  
SET zero_damaged_pages = on;  
SELECT * FROM my_table; -- recovers what it can, loses data on bad pages  
RESET zero_damaged_pages;  
  
-- Save recovered data to a new table  
CREATE TABLE my_table_recovered AS SELECT * FROM my_table;  
DROP TABLE my_table;  
ALTER TABLE my_table_recovered RENAME TO my_table;
```

Enable data checksums to detect future corruption:

bash

```
# PostgreSQL 12+: enable without reinitializing  
pg_ctl stop -D /var/lib/postgresql/14/main  
pg_checksums --enable -D /var/lib/postgresql/14/main  
pg_ctl start -D /var/lib/postgresql/14/main
```

Non-negotiable durability settings:

ini

```
# postgresql.conf
fsync = on                # NEVER set to off in production
synchronous_commit = on
```

Takeaway

Enable data checksums with `pg_checksums --enable` on all production PostgreSQL 12+ clusters. This catches disk corruption at read time before it propagates silently. Run `bt_index_check()` from `amcheck` periodically on critical indexes. `fsync = on` is non-negotiable in production — setting it to off to gain speed trades your data's durability for performance and will corrupt your entire cluster on any crash.

Closing: Building a Resilient PostgreSQL System

Every issue in this guide traces back to one of five root causes:

1. **Resource limits not tuned to workload** — `max_connections`, `work_mem`, `shared_buffers`, autovacuum workers
2. **Missing operational discipline** — no `ANALYZE` after bulk loads, no backup testing, no monitoring
3. **Schema design gaps** — missing FK indexes, wrong data types, unpartitioned time-series tables
4. **Reactive instead of proactive monitoring** — discovering replication lag, XID age, and connection exhaustion only during incidents
5. **Lock management failures** — long transactions, missing `lock_timeout`, no `idle_in_transaction_session_timeout`

All five are fixable with configuration, schema review, and metrics. PostgreSQL surfaces everything you need to know through `pg_stat_*` views — you just have to listen before things go wrong.

Quick Reference: Essential Monitoring Queries

```
sql

-- 1. Connection health
SELECT state, count(*) FROM pg_stat_activity
GROUP BY state ORDER BY count DESC;

-- 2. Blocking chains
SELECT blocked.pid, blocking.pid AS blocking_pid,
```

```

now() - blocked.query_start AS wait,
blocked.query AS waiting_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
    ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;

-- 3. XID wraparound risk (top 5)
SELECT relname, age(relfrozenxid) AS xid_age
FROM pg_class WHERE relkind = 'r'
ORDER BY xid_age DESC LIMIT 5;

-- 4. Autovacuum lag
SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 100000 ORDER BY n_dead_tup DESC LIMIT 10;

-- 5. Replication slot WAL retention
SELECT slot_name, active,
    pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS wal_retained
FROM pg_replication_slots ORDER BY 3 DESC;

-- 6. Database cache hit ratio
SELECT round(
    100.0 * sum(blks_hit) / nullif(sum(blks_hit + blks_read), 0), 2
) AS cache_hit_pct
FROM pg_stat_database
WHERE datname NOT IN ('template0', 'template1');

-- 7. Sequence exhaustion check (integer sequences)
SELECT s.relname AS sequence_name,
    round(100.0 * last_value / seqmax, 2) AS pct_used
FROM pg_class s
JOIN pg_sequence t ON t.seqrelid = s.oid
WHERE s.relkind = 'S' AND t.seqtupid = 'integer'::regtype::oid

ORDER BY pct_used DESC;

-- 8. Idle-in-transaction sessions
SELECT pid, username, now() - state_change AS idle_duration, left(query, 80)
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'idle in transaction (aborted)')
ORDER BY idle_duration DESC;

```

Parameter	Default	Recommended Start	Why
<code>shared_buffers</code>	128MB	25% of RAM	Page cache
<code>work_mem</code>	4MB	32-64MB	Sort/hash memory
<code>max_connections</code>	100	100 (+ pooler)	Use a pooler
<code>autovacuum_max_workers</code>	3	6	More parallel cleanup
<code>autovacuum_vacuum_scale_factor</code>	0.20	0.05	Vacuum sooner
<code>autovacuum_analyze_scale_factor</code>	0.10	0.02	Analyze sooner
<code>checkpoint_completion_target</code>	0.5	0.9	Spread I/O
<code>max_wal_size</code>	1GB	4-16GB	Reduce checkpoint storms
<code>idle_in_transaction_session_timeout</code>	0	5min	Kill idle-in-txn
<code>lock_timeout</code>	0	5-30s	Fail fast on lock waits
<code>log_lock_waits</code>	off	on	Visibility into contention
<code>log_min_duration_statement</code>	-1	500ms	Catch slow queries
<code>log_temp_files</code>	-1	100MB	Catch disk spills
<code>effective_cache_size</code>	4GB	75% of RAM	Planner hint
<code>wal_compression</code>	off	on	Reduce WAL size
<code>max_slot_wal_keep_size</code>	-1	10GB (PG 13+)	Cap slot WAL retention

Issue Severity Reference

#	Issue	Severity	Category
1	Connection exhaustion	 Critical	Connections
2	Lock contention / blocking chains	 Critical	Locks

3	Deadlocks	 Warning	Locks
4	Idle-in-transaction connections	 Warning	Locks
5	Transaction ID wraparound	 Critical	Vacuum
6	Autovacuum not keeping up	 Warning	Vacuum
7	Table bloat from dead tuples	 Warning	Vacuum
8	Index bloat	 Performance	Indexes
9	Missing indexes / seq scans	 Performance	Indexes
10	Unused indexes	 Performance	Indexes
11	Bad planner estimates	 Warning	Query
12	Stale statistics after bulk loads	 Warning	Query
13	work_mem too low / disk spills	 Performance	Memory
14	shared_buffers undersized	 Performance	Memory
15	Checkpoint storms	 Warning	I/O
16	WAL bloat filling disk	 Critical	WAL
17	Replication lag on standbys	 Critical	Replication
18	Inactive replication slots	 Critical	Replication
19	Hot standby conflict cancellation	 Warning	Replication
20	PgBouncer misconfiguration	 Warning	Connections
21	OOM kills	 Critical	Memory
22	Disk space exhaustion	 Critical	Operations
23	Temporary file explosion	 Warning	I/O
24	N+1 query patterns	 Performance	Query
25	Partition pruning not working	 Performance	Query
26	Parallel query not used	 Performance	Query
27	Plan regression after upgrade	 Warning	Query
28	Index not used (type mismatch)	 Performance	Indexes

29	Slow COUNT(*) on large tables	 Performance	Query
30	LIKE queries ignoring indexes	 Performance	Indexes
31	JSONB query performance	 Performance	Query
32	Full-text search performance	 Performance	Query
33	Sequence exhaustion	 Critical	Schema
34	Constraint violations at scale	 Warning	Schema
35	Foreign key locking overhead	 Warning	Schema
36	Long-running schema migrations	 Warning	Operations
37	Table lock from ALTER TABLE	 Warning	Operations
38	Connection storms after restart	 Warning	Connections
39	Logical replication lag/conflicts	 Warning	Replication
40	Backup failure from pg_dump	 Critical	Backups
41	PITR misconfiguration	 Critical	Backups
42	SSL/TLS connection issues	 Warning	Security
43	Extension conflicts	 Warning	Operations
44	Corrupt indexes and data pages	 Critical	Integrity

This guide covers PostgreSQL 13 through 16. Features and defaults vary by version. Always consult the official PostgreSQL documentation at <https://www.postgresql.org/docs/> for the version you are running.